

Mini Project Source Code

Blockchain Tourism Verification System

Generated: 2026-05-22 00:54

Included authored source/config files: 53

Safety Note

Secrets found in source files have been redacted in this PDF. Generated dependencies, build outputs, and the Google service-account credential JSON are excluded so this document remains focused on authored project code.

Included Files

backend/verificationApp/pom.xml
backend/verificationApp/src/main/java/com/miniproject/verificationApp/config/CorsConfig.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/config/SecurityConfig.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/controller/GovernmentVerificationController.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/controller/ReviewController.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/controller/UserController.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/EmailVerificationToken.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/GovernmentIdVerification.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/Review.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/ReviewFlag.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/User.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/LocationKeywordChecker.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/QualityResult.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/ReviewQualityService.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/SentimentAnalyzer.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/SpamDetector.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/EmailVerificationTokenRepository.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/GovernmentIdVerificationRepository.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/ReviewFlagRepository.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/ReviewRepository.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/UserRepository.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/AIVerificationResult.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/AIVerificationService.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/BlockchainService.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/ExifGpsService.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/HashUtil.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/OCRService.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/TimeDecayService.java
backend/verificationApp/src/main/java/com/miniproject/verificationApp/VerificationAppApplication.java
backend/verificationApp/src/main/resources/application.properties
backend/verificationApp/src/test/java/com/miniproject/verificationApp/VerificationAppApplicationTests.java
frontend/package.json
frontend/public/index.html
frontend/public/manifest.json
frontend/public/robots.txt
frontend/src/App.css
frontend/src/App.js
frontend/src/App.test.js
frontend/src/Dashboard.jsx
frontend/src/index.css
frontend/src/index.js
frontend/src/loginpage.jsx
frontend/src/logo.svg
frontend/src/MyReviews.jsx
frontend/src/PlaceReviews.jsx
frontend/src/registrationpage.jsx
frontend/src/reportWebVitals.js
frontend/src/Review.jsx
frontend/src/setupTests.js
frontend/src/TouristSelectPage.jsx
frontend/src/verificationpage.jsx
frontend/src/VerificationPage2.jsx
frontend/src/welcomepage.jsx

Omitted / Redacted

- backend/verificationApp/src/main/resources/google/blockchaintouristsystembASED.json - omitted because it is a Google service-account credential file containing private keys, not source code.
- frontend/node_modules/** - omitted because it is generated third-party dependency code.
- backend/verificationApp/target/** - omitted because it is generated build output.
- frontend/package-lock.json - omitted because it is generated dependency lock metadata, not authored code.

backend/verificationApp/pom.xml

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5 |     <modelVersion>4.0.0</modelVersion>
6 |
7 |     <parent>
8 |         <groupId>org.springframework.boot</groupId>
9 |         <artifactId>spring-boot-starter-parent</artifactId>
10 |         <version>3.5.7</version>
11 |         <relativePath/>
12 |     </parent>
13 |
14 |     <groupId>com.miniproject</groupId>
15 |     <artifactId>verificationApp</artifactId>
16 |     <version>0.0.1-SNAPSHOT</version>
17 |     <name>verificationApp</name>
18 |     <description>Government ID & Email Verification System</description>
19 |
20 |     <properties>
21 |         <java.version>21</java.version>
22 |     </properties>
23 |
24 |     <dependencies>
25 |         <!-- ■ Spring Boot Web Starter -->
26 |         <dependency>
27 |             <groupId>org.springframework.boot</groupId>
28 |             <artifactId>spring-boot-starter-web</artifactId>
29 |         </dependency>
30 |
31 |         <!-- ■ Spring Data JPA for database access -->
32 |         <dependency>
33 |             <groupId>org.springframework.boot</groupId>
34 |             <artifactId>spring-boot-starter-data-jpa</artifactId>
35 |         </dependency>
36 |
37 |         <!-- ■ Spring Security (for login/authorization) -->
38 |         <dependency>
39 |             <groupId>org.springframework.boot</groupId>
40 |             <artifactId>spring-boot-starter-security</artifactId>
41 |         </dependency>
42 |
43 |         <!-- ■ MySQL Driver -->
44 |         <dependency>
45 |             <groupId>com.mysql</groupId>
46 |             <artifactId>mysql-connector-j</artifactId>
47 |             <scope>runtime</scope>
48 |         </dependency>
49 |
50 |         <!-- ■ H2 Database (for testing or local development) -->
51 |         <dependency>
52 |             <groupId>com.h2database</groupId>
53 |             <artifactId>h2</artifactId>
54 |             <scope>runtime</scope>
55 |         </dependency>
56 |
57 |         <!-- ■ JavaMail (for email verification) -->
58 |         <dependency>
59 |             <groupId>com.sun.mail</groupId>
60 |             <artifactId>javax.mail</artifactId>
61 |             <version>1.6.2</version>
62 |         </dependency>
63 |
64 |         <!-- ■ File upload support -->
65 |         <dependency>
66 |             <groupId>commons-fileupload</groupId>
67 |             <artifactId>commons-fileupload</artifactId>
68 |             <version>1.5</version>
69 |         </dependency>
70 |
71 |         <!-- ■ OCR API HTTP client (fixes the TlsSocketStrategy error) -->
72 |         <dependency>
73 |             <groupId>org.apache.httpcomponents.client5</groupId>
74 |             <artifactId>httpclient5</artifactId>
75 |             <version>5.4</version>
76 |         </dependency>
77 |
78 |         <!-- ■ WebFlux (optional, for async/non-blocking requests) -->
79 |         <dependency>
80 |             <groupId>org.springframework.boot</groupId>
81 |             <artifactId>spring-boot-starter-webflux</artifactId>
82 |         </dependency>
83 |
84 |         <!-- ■ Dev Tools (auto reload in development) -->
85 |         <dependency>
86 |             <groupId>org.springframework.boot</groupId>
87 |             <artifactId>spring-boot-devtools</artifactId>
88 |             <scope>runtime</scope>
89 |             <optional>true</optional>

```

```
90 |         </dependency>
91 |
92 |         <!-- ■ Testing -->
93 |         <dependency>
94 |             <groupId>org.springframework.boot</groupId>
95 |             <artifactId>spring-boot-starter-test</artifactId>
96 |             <scope>test</scope>
97 |         </dependency>
98 |
99 |         <!-- ■ JSON Library -->
100 |        <dependency>
101 |            <groupId>org.json</groupId>
102 |            <artifactId>json</artifactId>
103 |            <version>20231013</version>
104 |        </dependency>
105 |        <!-- Existing dependencies stay as is -->
106 |
107 |    <!-- ADD THESE FOR GOOGLE VISION API -->
108 |        <dependency>
109 |            <groupId>com.google.cloud</groupId>
110 |            <artifactId>google-cloud-vision</artifactId>
111 |            <version>3.20.0</version>
112 |        </dependency>
113 |
114 |        <dependency>
115 |            <groupId>com.google.auth</groupId>
116 |            <artifactId>google-auth-library-oauth2-http</artifactId>
117 |            <version>1.19.0</version>
118 |        </dependency>
119 |
120 |        <!-- For JSON processing -->
121 |        <dependency>
122 |            <groupId>com.google.code.gson</groupId>
123 |            <artifactId>gson</artifactId>
124 |            <version>2.10.1</version>
125 |        </dependency>
126 |        <dependency>
127 |            <groupId>org.web3j</groupId>
128 |            <artifactId>core</artifactId>
129 |            <version>4.10.3</version>
130 |        </dependency>
131 |        <dependency>
132 |            <groupId>com.drewnoakes</groupId>
133 |            <artifactId>metadata-extractor</artifactId>
134 |            <version>2.19.0</version>
135 |        </dependency>
136 |        <dependency>
137 |            <groupId>edu.stanford.nlp</groupId>
138 |            <artifactId>stanford-corenlp</artifactId>
139 |            <version>4.5.6</version>
140 |        </dependency>
141 |
142 |        <dependency>
143 |            <groupId>edu.stanford.nlp</groupId>
144 |            <artifactId>stanford-corenlp</artifactId>
145 |            <version>4.5.6</version>
146 |            <classifier>models</classifier>
147 |        </dependency>
148 |
149 |    </dependencies>
150 |
151 |    <build>
152 |        <plugins>
153 |            <plugin>
154 |                <groupId>org.springframework.boot</groupId>
155 |                <artifactId>spring-boot-maven-plugin</artifactId>
156 |            </plugin>
157 |        </plugins>
158 |    </build>
159 | </project>
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/config/CorsConfig.java

```
1 | package com.miniproject.verificationApp.config;
2 |
3 | import org.springframework.context.annotation.Bean;
4 | import org.springframework.context.annotation.Configuration;
5 | import org.springframework.web.servlet.config.annotation.CorsRegistry;
6 | import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
7 |
8 | @Configuration
9 | public class CorsConfig {
10 |
11 |     @Bean
12 |     public WebMvcConfigurer corsConfigurer() {
13 |         return new WebMvcConfigurer() {
14 |             @Override
15 |             public void addCorsMappings(CorsRegistry registry) {
16 |                 registry.addMapping("/**") // allow all endpoints
17 |                     .allowedOrigins("http://localhost:3000") // allow React
18 |                     .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
19 |                     .allowedHeaders("*")
20 |                     .allowCredentials(true);
21 |             }
22 |         };
23 |     }
24 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/config/SecurityConfig.java

```
1 | package com.miniproject.verificationApp.config;
2 |
3 | import org.springframework.context.annotation.Bean;
4 | import org.springframework.context.annotation.Configuration;
5 | import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6 | import org.springframework.security.web.SecurityFilterChain;
7 | import org.springframework.web.cors.CorsConfiguration;
8 | import org.springframework.web.cors.CorsConfigurationSource;
9 | import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
10 |
11 | import java.util.List;
12 |
13 | @Configuration
14 | public class SecurityConfig {
15 |
16 |     @Bean
17 |     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
18 |         http
19 |             .csrf(csrf -> csrf.disable())
20 |             .cors(cors -> cors.configurationSource(corsConfigurationSource()));
21 |         return http.build();
22 |     }
23 |
24 |     @Bean
25 |     CorsConfigurationSource corsConfigurationSource() {
26 |         CorsConfiguration configuration = new CorsConfiguration();
27 |         configuration.setAllowedOrigins(List.of("http://localhost:3000"));
28 |         configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
29 |         configuration.setAllowedHeaders(List.of("*"));
30 |         configuration.setAllowCredentials(true);
31 |
32 |         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
33 |         source.registerCorsConfiguration("/**", configuration);
34 |         return source;
35 |     }
36 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/controller/GovernmentVerification

```

1 | package com.miniproject.verificationApp.controller;
2 |
3 | import com.miniproject.verificationApp.model.GovernmentIdVerification;
4 | import com.miniproject.verificationApp.model.User;
5 | import com.miniproject.verificationApp.repository.GovernmentIdVerificationRepository;
6 | import com.miniproject.verificationApp.repository.UserRepository;
7 | import com.miniproject.verificationApp.service.AIVerificationService;
8 | import com.miniproject.verificationApp.service.AIVerificationResult;
9 | import com.miniproject.verificationApp.service.ExifGpsService;
10 |
11 | import org.springframework.beans.factory.annotation.Autowired;
12 | import org.springframework.web.bind.annotation.*;
13 | import org.springframework.web.multipart.MultipartFile;
14 |
15 | import java.io.File;
16 | import java.io.IOException;
17 | import java.time.LocalDateTime;
18 | import java.util.*;
19 |
20 | @RestController
21 | @RequestMapping("/api/verification")
22 | @CrossOrigin(origins = "http://localhost:3000")
23 | public class GovernmentVerificationController {
24 |
25 |     @Autowired
26 |     private GovernmentIdVerificationRepository verificationRepository;
27 |
28 |     @Autowired
29 |     private UserRepository userRepository;
30 |
31 |     @Autowired
32 |     private AIVerificationService aiVerificationService;
33 |
34 |     @Autowired
35 |     private ExifGpsService exifGpsService;
36 |
37 |     private static final String UPLOAD_DIR = "C:/uploads/";
38 |
39 |     @PostMapping("/upload")
40 |     public String uploadProof(
41 |         @RequestParam("email") String email,
42 |         @RequestParam("file") MultipartFile file,
43 |         @RequestParam("selectedPlace") String selectedPlace,
44 |         @RequestParam("selectedLat") double selectedLat,
45 |         @RequestParam("selectedLon") double selectedLon
46 |     ) {
47 |
48 |         Optional<User> userOpt = userRepository.findByEmail(email);
49 |
50 |         if (userOpt.isEmpty()) {
51 |             return "ERROR:User not found";
52 |         }
53 |
54 |         User user = userOpt.get();
55 |
56 |         try {
57 |
58 |             File dir = new File(UPLOAD_DIR);
59 |
60 |             if (!dir.exists()) {
61 |                 dir.mkdirs();
62 |             }
63 |
64 |             String filePath =
65 |                 UPLOAD_DIR
66 |                 + System.currentTimeMillis()
67 |                 + "_"
68 |                 + file.getOriginalFilename();
69 |
70 |             File uploadedFile = new File(filePath);
71 |
72 |             file.transferTo(uploadedFile);
73 |
74 |             System.out.println("■ Analyzing image with AI...");
75 |
76 |             double[] gpsCoordinates =
77 |                 exifGpsService.extractGpsCoordinates(uploadedFile);
78 |
79 |             boolean gpsVerified = false;
80 |
81 |             double distanceMeters = -1;
82 |
83 |             Double photoLat = null;
84 |             Double photoLon = null;
85 |
86 |             String gpsReason = "";
87 |
88 |             if (gpsCoordinates != null) {
89 |

```

```
90 |         photoLat = gpsCoordinates[0];
91 |         photoLon = gpsCoordinates[1];
92 |
93 |         distanceMeters =
94 |             exifGpsService.calculateDistanceMeters(
95 |                 photoLat,
96 |                 photoLon,
97 |                 selectedLat,
98 |                 selectedLon
99 |             );
100 |
101 |         gpsVerified = distanceMeters <= 500;
102 |
103 |         System.out.println("■ Photo GPS: "
104 |             + photoLat + ", " + photoLon);
105 |
106 |         System.out.println("■ Distance: "
107 |             + distanceMeters + " meters");
108 |
109 |         if (!gpsVerified) {
110 |             gpsReason = "Photo GPS does not match selected location";
111 |             System.out.println("■ GPS Verification FAILED");
112 |         }
113 |
114 |     } else {
115 |
116 |         gpsReason = "No GPS metadata found";
117 |         System.out.println("■ No GPS metadata found");
118 |     }
119 |
120 |     AIVerificationResult aiResult =
121 |         aiVerificationService.verifyProof(
122 |             uploadedFile,
123 |             selectedPlace
124 |         );
125 |
126 |     GovernmentIdVerification verification =
127 |         new GovernmentIdVerification();
128 |
129 |     verification.setUser(user);
130 |     verification.setProofUrl(filePath);
131 |     verification.setPlaceName(selectedPlace);
132 |     verification.setCreatedAt(LocalDateTime.now());
133 |
134 |     verification.setPhotoLatitude(photoLat);
135 |     verification.setPhotoLongitude(photoLon);
136 |
137 |     verification.setGpsVerified(gpsVerified);
138 |     verification.setDistanceMeters(distanceMeters);
139 |
140 |     // ■ store AI confidence score
141 |     verification.setAiConfidenceScore(
142 |         (double) aiResult.getConfidenceScore()
143 |     );
144 |
145 |     if (aiResult.isVerified()
146 |         && (gpsVerified || gpsCoordinates == null)) {
147 |
148 |         verification.setStatus(
149 |             GovernmentIdVerification.Status.VERIFIED
150 |         );
151 |
152 |         verification.setVerifiedOn(LocalDateTime.now());
153 |
154 |         user.setGovernmentIdVerified(true);
155 |
156 |         userRepository.save(user);
157 |
158 |         verificationRepository.save(verification);
159 |
160 |         System.out.println("■ AI Verification SUCCESS: "
161 |             + aiResult.getMessage());
162 |
163 |         System.out.println("■ Confidence: "
164 |             + aiResult.getConfidenceScore() + "%");
165 |
166 |         System.out.println("■ Detected: "
167 |             + aiResult.getDetectedLandmark());
168 |
169 |         return String.format(
170 |             "VERIFIED:%s|CONFIDENCE:%.1f|TYPE:%s|GPS:%s|DISTANCE:%.2f",
171 |             selectedPlace,
172 |             aiResult.getConfidenceScore(),
173 |             aiResult.getVerificationType(),
174 |             gpsVerified,
175 |             distanceMeters
176 |         );
177 |     }
178 |
179 |     verification.setStatus(
180 |         GovernmentIdVerification.Status.REJECTED
```



```
181 |         );
182 |
183 |         verificationRepository.save(verification);
184 |
185 |         String reason;
186 |
187 |         if (!aiResult.isVerified()) {
188 |
189 |             reason =
190 |                 "AI could not verify this image. "
191 |                 + aiResult.getMessage();
192 |         }
193 |
194 |         else if (gpsCoordinates != null && !gpsVerified) {
195 |
196 |             reason =
197 |                 "GPS mismatch. Photo was taken "
198 |                 + String.format("%.2f", distanceMeters)
199 |                 + " meters away from selected place.";
200 |         }
201 |
202 |         else {
203 |
204 |             reason = "Verification failed";
205 |         }
206 |
207 |         System.out.println("■ Verification FAILED: " + reason);
208 |
209 |         return String.format(
210 |             "REJECTED:%s|AI_CONFIDENCE:%.1f|GPS_REASON:%s",
211 |             reason,
212 |             aiResult.getConfidenceScore(),
213 |             gpsReason
214 |         );
215 |
216 |     } catch (IOException e) {
217 |
218 |         e.printStackTrace();
219 |
220 |         return "ERROR:" + e.getMessage();
221 |     }
222 | }
223 |
224 | @GetMapping("/status")
225 | public List<GovernmentIdVerification> getVerificationStatus(
226 |     @RequestParam("email") String email
227 | ) {
228 |
229 |     Optional<User> userOpt = userRepository.findByEmail(email);
230 |
231 |     if (userOpt.isEmpty()) {
232 |         return new ArrayList<>();
233 |     }
234 |
235 |     User user = userOpt.get();
236 |
237 |     return verificationRepository.findById(user.getId());
238 | }
239 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/controller/ReviewController.java

```

1 | package com.miniproject.verificationApp.controller;
2 |
3 | import com.miniproject.verificationApp.model.*;
4 | import com.miniproject.verificationApp.repository.*;
5 | import com.miniproject.verificationApp.service.*;
6 | import com.miniproject.verificationApp.nlp.*;
7 |
8 | import org.springframework.beans.factory.annotation.Autowired;
9 | import org.springframework.http.ResponseEntity;
10 | import org.springframework.web.bind.annotation.*;
11 |
12 | import java.time.Duration;
13 | import java.time.LocalDateTime;
14 | import java.util.*;
15 |
16 | @RestController
17 | @RequestMapping("/api/reviews")
18 | @CrossOrigin(origins = "http://localhost:3000")
19 | public class ReviewController {
20 |
21 |     @Autowired
22 |     private TimeDecayService timeDecayService;
23 |
24 |     @Autowired
25 |     private ReviewFlagRepository reviewFlagRepository;
26 |
27 |     @Autowired
28 |     private ReviewRepository reviewRepository;
29 |
30 |     @Autowired
31 |     private UserRepository userRepository;
32 |
33 |     @Autowired
34 |     private BlockchainService blockchainService;
35 |
36 |     @Autowired
37 |     private ReviewQualityService reviewQualityService;
38 |
39 |     @Autowired
40 |     private GovernmentIdVerificationRepository verificationRepository;
41 |
42 |     // =====
43 |     // TRUST POINT CALCULATION
44 |     // =====
45 |     private double calculateTrustPoints(Review review) {
46 |
47 |         double aiScore = review.getAiConfidenceScore() == null
48 |             ? 0
49 |             : review.getAiConfidenceScore();
50 |
51 |         int flagCount = review.getFlagCount() == null
52 |             ? 0
53 |             : review.getFlagCount();
54 |
55 |         double disputeScore = Math.max(0, 100 - (flagCount * 25));
56 |
57 |         long daysOld = Duration.between(
58 |             review.getCreatedAt(),
59 |             LocalDateTime.now()
60 |         ).toDays();
61 |
62 |         double timeScore = Math.max(0, 100 - daysOld);
63 |
64 |         return (aiScore * 0.5)
65 |             + (timeScore * 0.3)
66 |             + (disputeScore * 0.2);
67 |     }
68 |
69 |     // =====
70 |     // ADD REVIEW
71 |     // =====
72 |     @PostMapping("/add")
73 |     public ResponseEntity<?> addReview(@RequestBody Map<String, String> request) {
74 |
75 |         String email = request.get("email");
76 |         String text = request.get("review");
77 |         String place = request.get("place");
78 |         String ratingStr = request.get("rating");
79 |
80 |         Integer rating = 1;
81 |
82 |         if (ratingStr != null && !ratingStr.isEmpty()) {
83 |             rating = Integer.parseInt(ratingStr);
84 |         }
85 |
86 |         User user = userRepository.findByEmail(email)
87 |             .orElseThrow(() -> new RuntimeException("User not found: " + email));
88 |
89 |         QualityResult result = reviewQualityService.analyze(text, place);

```

```

90 |
91 |         if (!result.isAccepted()) {
92 |
93 |             Map<String, Object> response = new HashMap<>();
94 |             response.put("status", "REJECTED");
95 |             response.put("qualityScore", result.getScore());
96 |             response.put("issues", result.getIssues());
97 |
98 |             return ResponseEntity.badRequest().body(response);
99 |         }
100 |
101 |         String hash = HashUtil.sha256(text + place + email);
102 |
103 |         Review review = new Review();
104 |         review.setUser(user);
105 |         review.setReviewText(text);
106 |         review.setPlaceName(place);
107 |         review.setReviewHash(hash);
108 |         review.setRating(rating);
109 |
110 |         Optional<GovernmentIdVerification> verificationOpt =
111 |             verificationRepository
112 |                 .findTopByUserIdAndPlaceNameAndStatusOrderByCreatedAtDesc(
113 |                     user.getId(),
114 |                     place,
115 |                     GovernmentIdVerification.Status.VERIFIED
116 |                 );
117 |
118 |         double aiConfidence = verificationOpt
119 |             .map(GovernmentIdVerification::getAiConfidenceScore)
120 |             .orElse(0.0);
121 |
122 |         review.setAiConfidenceScore(aiConfidence);
123 |
124 |         String txHash = blockchainService.storeHash(hash, place);
125 |
126 |         if (txHash == null) {
127 |             return ResponseEntity.status(500)
128 |                 .body("Blockchain storage failed. Review not saved.");
129 |         }
130 |
131 |         review.setStoredOnBlockchain(true);
132 |         review.setBlockchainTxHash(txHash);
133 |
134 |         // ■ Added here before calculating trust points
135 |         review.setCreatedAt(LocalDateTime.now());
136 |         review.setTrustPoints(calculateTrustPoints(review));
137 |
138 |         reviewRepository.save(review);
139 |
140 |         Map<String, Object> response = new HashMap<>();
141 |         response.put("status", "ACCEPTED");
142 |         response.put("qualityScore", result.getScore());
143 |         response.put("aiConfidence", aiConfidence);
144 |         response.put("trustPoints", review.getTrustPoints());
145 |         response.put("txHash", txHash);
146 |         response.put("message", "Review stored with blockchain proof!");
147 |
148 |         return ResponseEntity.ok(response);
149 |     }
150 |
151 |     // =====
152 |     // FLAG REVIEW
153 |     // =====
154 |     @PostMapping("/flag")
155 |     public ResponseEntity<?> flagReview(@RequestBody Map<String, String> request) {
156 |
157 |         try {
158 |             Long reviewId = Long.parseLong(request.get("reviewId"));
159 |             String email = request.get("email");
160 |
161 |             User user = userRepository.findById(email)
162 |                 .orElseThrow(() -> new RuntimeException("User not found"));
163 |
164 |             Review review = reviewRepository.findById(reviewId)
165 |                 .orElseThrow(() -> new RuntimeException("Review not found"));
166 |
167 |             if (user.getGovernmentIdVerified() == null || !user.getGovernmentIdVerified()) {
168 |                 return ResponseEntity.badRequest().body("Only verified users can flag");
169 |             }
170 |
171 |             if (reviewFlagRepository.existsByReviewIdAndUserId(reviewId, user.getId())) {
172 |                 return ResponseEntity.badRequest().body("Already flagged");
173 |             }
174 |
175 |             ReviewFlag flag = new ReviewFlag(reviewId, user.getId());
176 |             reviewFlagRepository.save(flag);
177 |
178 |             int currentCount = review.getFlagCount() == null ? 0 : review.getFlagCount();
179 |             review.setFlagCount(currentCount + 1);
180 |

```

```
181 |         if (review.getFlagCount() >= 3) {
182 |             review.setIsDisputed(true);
183 |         }
184 |
185 |         review.setTrustPoints(calculateTrustPoints(review));
186 |
187 |         reviewRepository.save(review);
188 |
189 |         return ResponseEntity.ok("Flag added successfully");
190 |
191 |     } catch (Exception e) {
192 |         e.printStackTrace();
193 |         return ResponseEntity.status(500).body("Error: " + e.getMessage());
194 |     }
195 | }
196 |
197 | // =====
198 | // GET ALL REVIEWS
199 | // =====
200 | @GetMapping("/all")
201 | public List<Review> getAll() {
202 |     return reviewRepository.findAll();
203 | }
204 |
205 | // =====
206 | // GET MY REVIEWS
207 | // =====
208 | @GetMapping("/my-reviews")
209 | public List<Review> getMyReviews(@RequestParam String email) {
210 |
211 |     User user = userRepository.findByEmail(email)
212 |         .orElseThrow(() -> new RuntimeException("User not found"));
213 |
214 |     return reviewRepository.findByIdAndStoredOnBlockchainTrue(user.getId());
215 | }
216 |
217 | // =====
218 | // SEARCH REVIEWS BY PLACE
219 | // =====
220 | @GetMapping("/search")
221 | public List<Review> searchReviews(@RequestParam String place) {
222 |
223 |     return reviewRepository
224 |         .findByPlaceNameContainingIgnoreCaseAndStoredOnBlockchainTrueOrderByTrustPointsDesc(place);
225 | }
226 |
227 | // =====
228 | // RATINGS
229 | // =====
230 | @GetMapping("/rating")
231 | public Map<String, Double> getRatings(@RequestParam String place) {
232 |
233 |     List<Review> reviews = reviewRepository.findByPlaceName(place);
234 |
235 |     double rawRating = timeDecayService.calculateRawRating(reviews);
236 |     double decayedRating = timeDecayService.calculateDecayedRating(reviews);
237 |
238 |     Map<String, Double> response = new HashMap<>();
239 |     response.put("rawRating", rawRating);
240 |     response.put("decayedRating", decayedRating);
241 |
242 |     return response;
243 | }
244 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/controller/UserController.java

```
1 | package com.miniproject.verificationApp.controller;
2 |
3 | import com.miniproject.verificationApp.model.User;
4 | import com.miniproject.verificationApp.model.EmailVerificationToken;
5 | import com.miniproject.verificationApp.repository.UserRepository;
6 | import com.miniproject.verificationApp.repository.EmailVerificationTokenRepository;
7 |
8 | import org.springframework.beans.factory.annotation.Autowired;
9 | import org.springframework.http.HttpStatus;
10 | import org.springframework.http.ResponseEntity;
11 | import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
12 | import org.springframework.web.bind.annotation.*;
13 |
14 | import java.security.MessageDigest;
15 | import java.security.NoSuchAlgorithmException;
16 | import java.time.LocalDateTime;
17 | import java.util.*;
18 | import javax.mail.*;
19 | import javax.mail.internet.*;
20 |
21 | @RestController
22 | @RequestMapping("/api")
23 | @CrossOrigin(origins = "http://localhost:3000")
24 | public class UserController {
25 |
26 |     @Autowired
27 |     private UserRepository userRepository;
28 |
29 |     @Autowired
30 |     private EmailVerificationTokenRepository tokenRepository;
31 |
32 |     private final BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();
33 |
34 |     // ■ Registration
35 |     @PostMapping("/register")
36 |     public String register(@RequestBody User user) {
37 |         Optional<User> existing = userRepository.findByEmail(user.getEmail());
38 |         if (existing.isPresent()) {
39 |             return "Email already registered";
40 |         }
41 |
42 |         // Hash password before saving
43 |         user.setPasswordHash(passwordEncoder.encode(user.getPasswordHash()));
44 |         user.setIsActive(true);
45 |         user.setEmailVerified(false);
46 |         userRepository.save(user);
47 |         return "Registration successful! Verify your email.";
48 |     }
49 |
50 |     // ■ Login (Updated to return JSON)
51 |     @PostMapping("/login")
52 |     public ResponseEntity<?> login(@RequestBody User loginRequest) {
53 |         Optional<User> userOpt = userRepository.findByEmail(loginRequest.getEmail());
54 |         if (userOpt.isEmpty()) {
55 |             return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
56 |                 .body(Map.of("message", "No account found"));
57 |         }
58 |
59 |         User user = userOpt.get();
60 |
61 |         // Compare password hash
62 |         if (!passwordEncoder.matches(loginRequest.getPasswordHash(), user.getPasswordHash())) {
63 |             return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
64 |                 .body(Map.of("message", "Invalid password"));
65 |         }
66 |
67 |         if (!user.getIsActive()) {
68 |             return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
69 |                 .body(Map.of("message", "Account is inactive"));
70 |         }
71 |
72 |         // ■ JSON Response with email verification status
73 |         Map<String, Object> response = new HashMap<>();
74 |         response.put("message", "Login successful!");
75 |         response.put("emailVerified", user.getEmailVerified());
76 |
77 |         return ResponseEntity.ok(response);
78 |     }
79 |
80 |     // ■ Send verification token
81 |     @PostMapping("/send-token")
82 |     public String sendToken(@RequestBody EmailRequest request) {
83 |         Optional<User> userOpt = userRepository.findByEmail(request.getEmail());
84 |         if (userOpt.isEmpty()) return "Email not registered";
85 |
86 |         User user = userOpt.get();
87 |
88 |         String token = UUID.randomUUID().toString();
89 |         String tokenHash = hash(token);
```

```
90 |         LocalDateTime expiry = LocalDateTime.now().plusMinutes(30);
91 |
92 |         EmailVerificationToken emailToken = new EmailVerificationToken();
93 |         emailToken.setUser(user);
94 |         emailToken.setTokenHash(tokenHash);
95 |         emailToken.setExpiresAt(expiry);
96 |         tokenRepository.save(emailToken);
97 |
98 |         try {
99 |             sendEmail(user.getEmail(), token);
100 |             return "Verification token sent to your email!";
101 |         } catch (Exception e) {
102 |             e.printStackTrace();
103 |             return "Failed to send email: " + e.getMessage();
104 |         }
105 |     }
106 |
107 |     // ■ Verify token
108 |     @PostMapping("/verify-token")
109 |     public String verifyToken(@RequestBody VerificationRequest request) {
110 |         Optional<User> userOpt = userRepository.findByEmail(request.getEmail());
111 |         if (userOpt.isEmpty()) return "Email not registered";
112 |
113 |         User user = userOpt.get();
114 |
115 |         Optional<EmailVerificationToken> tokenOpt = tokenRepository.findTopByUserOrderByCreatedAtDesc(user);
116 |         if (tokenOpt.isEmpty()) return "No token found. Request a token first.";
117 |
118 |         EmailVerificationToken tokenEntity = tokenOpt.get();
119 |
120 |         if (tokenEntity.getExpiresAt().isBefore(LocalDateTime.now())) return "Token expired!";
121 |         if (!hash(request.getToken()).equals(tokenEntity.getTokenHash())) return "Invalid token!";
122 |
123 |         user.setEmailVerified(true);
124 |         userRepository.save(user);
125 |
126 |         return "Email verified successfully!";
127 |     }
128 |
129 |     // ■ Helper: Hash function
130 |     private String hash(String input) {
131 |         try {
132 |             MessageDigest md = MessageDigest.getInstance("SHA-256");
133 |             byte[] hash = md.digest(input.getBytes());
134 |             StringBuilder sb = new StringBuilder();
135 |             for (byte b : hash) sb.append(String.format("%02x", b));
136 |             return sb.toString();
137 |         } catch (NoSuchAlgorithmException e) {
138 |             e.printStackTrace();
139 |             return null;
140 |         }
141 |     }
142 |
143 |     // ■ Helper: Send Email
144 |     private void sendEmail(String toEmail, String token) throws MessagingException {
145 |         String fromEmail = "veri10fication@gmail.com";
146 |         String fromPassword = [REDACTED_PASSWORD]; // Gmail App password
147 |
148 |         Properties props = new Properties();
149 |         props.put("mail.smtp.auth", "true");
150 |         props.put("mail.smtp.starttls.enable", "true");
151 |         props.put("mail.smtp.host", "smtp.gmail.com");
152 |         props.put("mail.smtp.port", "587");
153 |
154 |         Session session = Session.getInstance(props, new Authenticator() {
155 |             protected PasswordAuthentication getPasswordAuthentication() {
156 |                 return new PasswordAuthentication(fromEmail, fromPassword);
157 |             }
158 |         });
159 |
160 |         Message message = new MimeMessage(session);
161 |         message.setFrom(new InternetAddress(fromEmail));
162 |         message.setRecipients(Message.RecipientType.TO, InternetAddress.parse(toEmail));
163 |         message.setSubject("Your Verification Token");
164 |         message.setText("Hello,\n\nYour verification token is: " + token +
165 |             "\n\nThis token expires in 30 minutes.\n\nThank you.");
166 |         Transport.send(message);
167 |     }
168 |
169 |     // ■ DTOs
170 |     public static class EmailRequest {
171 |         private String email;
172 |         public String getEmail() { return email; }
173 |         public void setEmail(String email) { this.email = email; }
174 |     }
175 |
176 |     public static class VerificationRequest {
177 |         private String email;
178 |         private String token;
179 |         public String getEmail() { return email; }
180 |         public void setEmail(String email) { this.email = email; }
```

```
181 |         public String getToken() { return token; }
182 |         public void setToken(String token) { this.token = token; }
183 |     }
184 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/EmailVerificationToken.java

```
1 | package com.miniproject.verificationApp.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDateTime;
5 |
6 | @Entity
7 | @Table(name = "email_verification_tokens")
8 | public class EmailVerificationToken {
9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     private Long id;
13 |
14 |     @ManyToOne
15 |     @JoinColumn(name = "user_id", nullable = false)
16 |     private User user;
17 |
18 |     @Column(name = "token_hash", nullable = false)
19 |     private String tokenHash;
20 |
21 |     @Column(name = "expires_at", nullable = false)
22 |     private LocalDateTime expiresAt;
23 |
24 |     @Column(name = "created_at", nullable = false)
25 |     private LocalDateTime createdAt = LocalDateTime.now();
26 |
27 |     // Getters and setters
28 |     public Long getId() { return id; }
29 |     public User getUser() { return user; }
30 |     public void setUser(User user) { this.user = user; }
31 |     public String getTokenHash() { return tokenHash; }
32 |     public void setTokenHash(String tokenHash) { this.tokenHash = tokenHash; }
33 |     public LocalDateTime getExpiresAt() { return expiresAt; }
34 |     public void setExpiresAt(LocalDateTime expiresAt) { this.expiresAt = expiresAt; }
35 |     public LocalDateTime getCreatedAt() { return createdAt; }
36 |     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
37 | }
```


backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/GovernmentIdVerification.j

```
1 | package com.miniproject.verificationApp.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDateTime;
5 |
6 | @Entity
7 | @Table(name = "government_id_verifications")
8 | public class GovernmentIdVerification {
9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     private Long id;
13 |
14 |     // Foreign key to users table
15 |     @ManyToOne
16 |     @JoinColumn(name = "user_id", nullable = false)
17 |     private User user;
18 |
19 |     @Enumerated(EnumType.STRING)
20 |     @Column(nullable = false)
21 |     private Status status = Status.PENDING;
22 |
23 |     @Column(name = "proof_url", length = 1024)
24 |     private String proofUrl;
25 |
26 |     @Column(name = "ai_confidence_score")
27 |     private Double aiConfidenceScore;
28 |
29 |     @Column(name = "verified_on")
30 |     private LocalDateTime verifiedOn;
31 |
32 |     @Column(name = "created_at")
33 |     private LocalDateTime createdAt = LocalDateTime.now();
34 |
35 |     @Column(name = "place_name")
36 |     private String placeName;
37 |
38 |     @Column(name = "photo_latitude")
39 |     private Double photoLatitude;
40 |
41 |     @Column(name = "photo_longitude")
42 |     private Double photoLongitude;
43 |
44 |     @Column(name = "gps_verified")
45 |     private Boolean gpsVerified = false;
46 |
47 |     @Column(name = "distance_meters")
48 |     private Double distanceMeters;
49 |
50 |     public enum Status {
51 |         PENDING, VERIFIED, REJECTED
52 |     }
53 |
54 |     public Long getId() {
55 |         return id;
56 |     }
57 |
58 |     public void setId(Long id) {
59 |         this.id = id;
60 |     }
61 |
62 |     public User getUser() {
63 |         return user;
64 |     }
65 |
66 |     public void setUser(User user) {
67 |         this.user = user;
68 |     }
69 |
70 |     public Double getAiConfidenceScore() {
71 |         return aiConfidenceScore;
72 |     }
73 |
74 |     public void setAiConfidenceScore(Double aiConfidenceScore) {
75 |         this.aiConfidenceScore = aiConfidenceScore;
76 |     }
77 |
78 |     public Status getStatus() {
79 |         return status;
80 |     }
81 |
82 |     public void setStatus(Status status) {
83 |         this.status = status;
84 |     }
85 |
86 |     public String getProofUrl() {
87 |         return proofUrl;
88 |     }
89 | }
```

```
90 |     public void setProofUrl(String proofUrl) {
91 |         this.proofUrl = proofUrl;
92 |     }
93 |
94 |     public LocalDateTime getVerifiedOn() {
95 |         return verifiedOn;
96 |     }
97 |
98 |     public void setVerifiedOn(LocalDateTime verifiedOn) {
99 |         this.verifiedOn = verifiedOn;
100 |     }
101 |
102 |     public LocalDateTime getCreatedAt() {
103 |         return createdAt;
104 |     }
105 |
106 |     public void setCreatedAt(LocalDateTime createdAt) {
107 |         this.createdAt = createdAt;
108 |     }
109 |
110 |     public String getPlaceName() {
111 |         return placeName;
112 |     }
113 |
114 |     public void setPlaceName(String placeName) {
115 |         this.placeName = placeName;
116 |     }
117 |
118 |     public Double getPhotoLatitude() {
119 |         return photoLatitude;
120 |     }
121 |
122 |     public void setPhotoLatitude(Double photoLatitude) {
123 |         this.photoLatitude = photoLatitude;
124 |     }
125 |
126 |     public Double getPhotoLongitude() {
127 |         return photoLongitude;
128 |     }
129 |
130 |     public void setPhotoLongitude(Double photoLongitude) {
131 |         this.photoLongitude = photoLongitude;
132 |     }
133 |
134 |     public Boolean getGpsVerified() {
135 |         return gpsVerified;
136 |     }
137 |
138 |     public void setGpsVerified(Boolean gpsVerified) {
139 |         this.gpsVerified = gpsVerified;
140 |     }
141 |
142 |     public Double getDistanceMeters() {
143 |         return distanceMeters;
144 |     }
145 |
146 |     public void setDistanceMeters(Double distanceMeters) {
147 |         this.distanceMeters = distanceMeters;
148 |     }
149 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/Review.java

```
1 | package com.miniproject.verificationApp.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDateTime;
5 |
6 | @Entity
7 | @Table(name = "reviews")
8 | public class Review {
9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     private Long id;
13 |
14 |     @ManyToOne
15 |     @JoinColumn(name = "user_id")
16 |     private User user;
17 |
18 |     @Column(name = "flag_count")
19 |     private Integer flagCount = 0;
20 |
21 |     @Column(name = "is_disputed")
22 |     private Boolean isDisputed = false;
23 |
24 |     @Column(columnDefinition = "TEXT")
25 |     private String reviewText;
26 |
27 |     private String placeName;
28 |
29 |     private String reviewHash;
30 |
31 |     private Integer rating;
32 |
33 |     private LocalDateTime createdAt = LocalDateTime.now();
34 |
35 |     @Column(name = "stored_on_blockchain")
36 |     private Boolean storedOnBlockchain = false;
37 |
38 |     @Column(name = "blockchain_tx_hash")
39 |     private String blockchainTxHash;
40 |
41 |     @Column(name = "ai_confidence_score")
42 |     private Double aiConfidenceScore = 0.0;
43 |
44 |     @Column(name = "trust_points")
45 |     private Double trustPoints = 0.0;
46 |
47 |     public Long getId() {
48 |         return id;
49 |     }
50 |
51 |     public User getUser() {
52 |         return user;
53 |     }
54 |
55 |     public void setUser(User user) {
56 |         this.user = user;
57 |     }
58 |
59 |     public String getReviewText() {
60 |         return reviewText;
61 |     }
62 |
63 |     public void setReviewText(String reviewText) {
64 |         this.reviewText = reviewText;
65 |     }
66 |
67 |     public String getPlaceName() {
68 |         return placeName;
69 |     }
70 |
71 |     public void setPlaceName(String placeName) {
72 |         this.placeName = placeName;
73 |     }
74 |
75 |     public String getReviewHash() {
76 |         return reviewHash;
77 |     }
78 |
79 |     public void setReviewHash(String reviewHash) {
80 |         this.reviewHash = reviewHash;
81 |     }
82 |
83 |     public Integer getRating() {
84 |         return rating;
85 |     }
86 |
87 |     public void setRating(Integer rating) {
88 |         this.rating = rating;
89 |     }
90 | }
```

```
90 |  
91 |     public Integer getFlagCount() {  
92 |         return flagCount == null ? 0 : flagCount;  
93 |     }  
94 |  
95 |     public void setFlagCount(Integer flagCount) {  
96 |         this.flagCount = flagCount;  
97 |     }  
98 |  
99 |     public Boolean getIsDisputed() {  
100 |         return isDisputed != null && isDisputed;  
101 |     }  
102 |  
103 |     public void setIsDisputed(Boolean isDisputed) {  
104 |         this.isDisputed = isDisputed;  
105 |     }  
106 |     public void setCreatedAt(LocalDateTime createdAt) {  
107 |         this.createdAt = createdAt;  
108 |     }  
109 |     public LocalDateTime getCreatedAt() {  
110 |         return createdAt;  
111 |     }  
112 |  
113 |     public Boolean getStoredOnBlockchain() {  
114 |         return storedOnBlockchain != null && storedOnBlockchain;  
115 |     }  
116 |  
117 |     public void setStoredOnBlockchain(Boolean storedOnBlockchain) {  
118 |         this.storedOnBlockchain = storedOnBlockchain;  
119 |     }  
120 |  
121 |     public String getBlockchainTxHash() {  
122 |         return blockchainTxHash;  
123 |     }  
124 |  
125 |     public void setBlockchainTxHash(String blockchainTxHash) {  
126 |         this.blockchainTxHash = blockchainTxHash;  
127 |     }  
128 |  
129 |     public Double getAiConfidenceScore() {  
130 |         return aiConfidenceScore == null ? 0.0 : aiConfidenceScore;  
131 |     }  
132 |  
133 |     public void setAiConfidenceScore(Double aiConfidenceScore) {  
134 |         this.aiConfidenceScore = aiConfidenceScore;  
135 |     }  
136 |  
137 |     public Double getTrustPoints() {  
138 |         return trustPoints == null ? 0.0 : trustPoints;  
139 |     }  
140 |  
141 |     public void setTrustPoints(Double trustPoints) {  
142 |         this.trustPoints = trustPoints;  
143 |     }  
144 |  
145 |     @PrePersist  
146 |     public void onCreate() {  
147 |         this.createdAt = LocalDateTime.now();  
148 |     }  
149 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/ReviewFlag.java

```
1 | package com.miniproject.verificationApp.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | @Entity
6 | @Table(name = "review_flags")
7 | public class ReviewFlag {
8 |
9 |     @Id
10 |    @GeneratedValue(strategy = GenerationType.IDENTITY)
11 |    private Long id;
12 |
13 |    private Long reviewId;
14 |
15 |    private Long userId;
16 |
17 |    public ReviewFlag() {}
18 |
19 |    public ReviewFlag(Long reviewId, Long userId) {
20 |        this.reviewId = reviewId;
21 |        this.userId = userId;
22 |    }
23 |
24 |    public Long getId() {
25 |        return id;
26 |    }
27 |
28 |    public Long getReviewId() {
29 |        return reviewId;
30 |    }
31 |
32 |    public void setReviewId(Long reviewId) {
33 |        this.reviewId = reviewId;
34 |    }
35 |
36 |    public Long getUserId() {
37 |        return userId;
38 |    }
39 |
40 |    public void setUserId(Long userId) {
41 |        this.userId = userId;
42 |    }
43 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/model/User.java

```
1 | package com.miniproject.verificationApp.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDateTime;
5 |
6 | @Entity
7 | @Table(name = "users") // matches your existing table name
8 | public class User {
9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     @Column(name = "user_id")
13 |     private Long id;
14 |
15 |     @Column(name = "name", nullable = false, length = 150)
16 |     private String name;
17 |
18 |     @Column(name = "email", nullable = false, unique = true, length = 255)
19 |     private String email;
20 |
21 |     @Column(name = "password_hash", nullable = false, length = 255)
22 |     private String passwordHash;
23 |
24 |     @Column(name = "email_verified", nullable = false)
25 |     private Boolean emailVerified = false;
26 |
27 |     @Column(name = "government_id_verified", nullable = false)
28 |     private Boolean governmentIdVerified = false;
29 |
30 |     @Column(name = "date_registered", nullable = false)
31 |     private LocalDateTime dateRegistered = LocalDateTime.now();
32 |
33 |     @Column(name = "is_active", nullable = false)
34 |     private Boolean isActive = true;
35 |
36 |     // Getters and Setters
37 |
38 |     public Long getId() {
39 |         return id;
40 |     }
41 |
42 |     public void setId(Long id) {
43 |         this.id = id;
44 |     }
45 |
46 |     public String getName() {
47 |         return name;
48 |     }
49 |
50 |     public void setName(String name) {
51 |         this.name = name;
52 |     }
53 |
54 |     public String getEmail() {
55 |         return email;
56 |     }
57 |
58 |     public void setEmail(String email) {
59 |         this.email = email;
60 |     }
61 |
62 |     public String getPasswordHash() {
63 |         return passwordHash;
64 |     }
65 |
66 |     public void setPasswordHash(String passwordHash) {
67 |         this.passwordHash = passwordHash;
68 |     }
69 |
70 |     public Boolean getEmailVerified() {
71 |         return emailVerified;
72 |     }
73 |
74 |     public void setEmailVerified(Boolean emailVerified) {
75 |         this.emailVerified = emailVerified;
76 |     }
77 |
78 |     public Boolean getGovernmentIdVerified() {
79 |         return governmentIdVerified;
80 |     }
81 |
82 |     public void setGovernmentIdVerified(Boolean governmentIdVerified) {
83 |         this.governmentIdVerified = governmentIdVerified;
84 |     }
85 |
86 |     public LocalDateTime getDateRegistered() {
87 |         return dateRegistered;
88 |     }
89 | }
```

```
90 |     public void setDateRegistered(LocalDateTime dateRegistered) {
91 |         this.dateRegistered = dateRegistered;
92 |     }
93 |
94 |     public Boolean getIsActive() {
95 |         return isActive;
96 |     }
97 |
98 |     public void setIsActive(Boolean isActive) {
99 |         this.isActive = isActive;
100 |     }
101 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/LocationKeywordChecker.java

```
1 | package com.miniproject.verificationApp.nlp;
2 |
3 | public class LocationKeywordChecker {
4 |
5 |     public boolean containsLocation(String review, String place) {
6 |
7 |         review = review.toLowerCase();
8 |         place = place.toLowerCase();
9 |
10 |        // Exact full place match
11 |        if(review.contains(place)) {
12 |            return true;
13 |        }
14 |
15 |        // Split place words and check individually
16 |        String[] placeWords = place.split("\\s+");
17 |
18 |        int matchedWords = 0;
19 |
20 |        for(String word : placeWords) {
21 |
22 |            if(word.length() < 3) {
23 |                continue;
24 |            }
25 |
26 |            if(review.contains(word)) {
27 |                matchedWords++;
28 |            }
29 |        }
30 |
31 |        // Require at least half the place words
32 |        return matchedWords >= Math.max(1, placeWords.length / 2);
33 |    }
34 | }
```


backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/QualityResult.java

```
1 | package com.miniproject.verificationApp.nlp;
2 |
3 | import java.util.List;
4 |
5 | public class QualityResult {
6 |
7 |     private int score;
8 |     private boolean accepted;
9 |     private List<String> issues;
10 |
11 |     public QualityResult(int score, boolean accepted, List<String> issues) {
12 |         this.score = score;
13 |         this.accepted = accepted;
14 |         this.issues = issues;
15 |     }
16 |
17 |     public int getScore() {
18 |         return score;
19 |     }
20 |
21 |     public boolean isAccepted() {
22 |         return accepted;
23 |     }
24 |
25 |     public List<String> getIssues() {
26 |         return issues;
27 |     }
28 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/ReviewQualityService.java

```
1 | package com.miniproject.verificationApp.nlp;
2 |
3 | import org.springframework.stereotype.Service;
4 |
5 | import java.util.ArrayList;
6 | import java.util.List;
7 |
8 | @Service
9 | public class ReviewQualityService {
10 |
11 |     private final SpamDetector spamDetector =
12 |         new SpamDetector();
13 |
14 |     private final SentimentAnalyzer sentimentAnalyzer =
15 |         new SentimentAnalyzer();
16 |
17 |     private final LocationKeywordChecker locationChecker =
18 |         new LocationKeywordChecker();
19 |
20 |     public QualityResult analyze(String review, String place) {
21 |
22 |         int score = 0;
23 |
24 |         List<String> issues = new ArrayList<>();
25 |
26 |         // =====
27 |         // WORD COUNT CHECK
28 |         // =====
29 |
30 |         int wordCount = review.trim().split("\\s+").length;
31 |
32 |         if(wordCount >= 15) {
33 |             score += 30;
34 |         } else {
35 |             issues.add("Review too short (minimum 15 words required)");
36 |         }
37 |
38 |         // =====
39 |         // SPAM CHECK
40 |         // =====
41 |
42 |         boolean spam = spamDetector.isSpam(review);
43 |
44 |         if(!spam) {
45 |             score += 20;
46 |         } else {
47 |             issues.add("Spam-like review detected");
48 |         }
49 |
50 |         // =====
51 |         // SENTIMENT CHECK
52 |         // =====
53 |
54 |         String sentiment =
55 |             sentimentAnalyzer.analyzeSentiment(review);
56 |
57 |         if(!sentiment.equalsIgnoreCase("Neutral")) {
58 |             score += 20;
59 |         } else {
60 |             issues.add("Weak or neutral sentiment detected");
61 |         }
62 |
63 |         // =====
64 |         // LOCATION CHECK
65 |         // =====
66 |
67 |         boolean containsLocation =
68 |             locationChecker.containsLocation(review, place);
69 |
70 |         if(containsLocation) {
71 |             score += 30;
72 |         } else {
73 |             issues.add("Tourist location not clearly mentioned");
74 |         }
75 |
76 |         // =====
77 |         // FINAL DECISION
78 |         // =====
79 |
80 |         boolean accepted = score >= 70;
81 |
82 |         return new QualityResult(
83 |             score,
84 |             accepted,
85 |             issues
86 |         );
87 |     }
88 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/SentimentAnalyzer.java

```
1 | package com.miniproject.verificationApp.nlp;
2 |
3 | import edu.stanford.nlp.pipeline.*;
4 |
5 | import java.util.Properties;
6 |
7 | public class SentimentAnalyzer {
8 |
9 |     private final StanfordCoreNLP pipeline;
10 |
11 |     public SentimentAnalyzer() {
12 |
13 |         Properties props = new Properties();
14 |
15 |         props.setProperty(
16 |             "annotators",
17 |             "tokenize,ssplit,pos,parse,sentiment"
18 |         );
19 |
20 |         pipeline = new StanfordCoreNLP(props);
21 |     }
22 |
23 |     public String analyzeSentiment(String text) {
24 |
25 |         CoreDocument document = new CoreDocument(text);
26 |
27 |         pipeline.annotate(document);
28 |
29 |         if (document.sentences().isEmpty()) {
30 |             return "Neutral";
31 |         }
32 |
33 |         // ■ Correct method for CoreNLP 4.5.6
34 |         return document.sentences()
35 |             .get(0)
36 |             .sentiment();
37 |     }
38 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/nlp/SpamDetector.java

```
1 | package com.miniproject.verificationApp.nlp;
2 |
3 | import java.util.HashSet;
4 | import java.util.Set;
5 |
6 | public class SpamDetector {
7 |
8 |     public boolean isSpam(String review) {
9 |
10 |         String lower = review.toLowerCase();
11 |
12 |         // =====
13 |         // REPEATED WORDS
14 |         // =====
15 |
16 |         if(lower.matches(".*(\\b\\w+\\b)(\\s+\\1\\b){2,}.*)") {
17 |             return true;
18 |         }
19 |
20 |         // =====
21 |         // TOO MANY CAPITALS
22 |         // =====
23 |
24 |         long upperCount = review.chars()
25 |             .filter(Character::isUpperCase)
26 |             .count();
27 |
28 |         if(review.length() > 0 &&
29 |             (upperCount * 100 / review.length()) > 40) {
30 |             return true;
31 |         }
32 |
33 |         // =====
34 |         // EXCESSIVE PUNCTUATION
35 |         // =====
36 |
37 |         if(review.matches(".*[!?.]{4,}.*)") {
38 |             return true;
39 |         }
40 |
41 |         // =====
42 |         // TOO MANY DUPLICATE WORDS
43 |         // =====
44 |
45 |         String[] words = lower.split("\\s+");
46 |
47 |         Set<String> uniqueWords = new HashSet<>();
48 |
49 |         for(String word : words) {
50 |             uniqueWords.add(word);
51 |         }
52 |
53 |         if(words.length > 0 &&
54 |             uniqueWords.size() < words.length / 2) {
55 |             return true;
56 |         }
57 |
58 |         return false;
59 |     }
60 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/EmailVerificationToken

```
1 | package com.miniproject.verificationApp.repository;
2 |
3 | import com.miniproject.verificationApp.model.EmailVerificationToken;
4 | import com.miniproject.verificationApp.model.User;
5 | import org.springframework.data.jpa.repository.JpaRepository;
6 |
7 | import java.util.Optional;
8 |
9 | public interface EmailVerificationTokenRepository extends JpaRepository<EmailVerificationToken, Long> {
10 |     Optional<EmailVerificationToken> findTopByUserOrderByCreatedAtDesc(User user);
11 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/GovernmentIdVerificati

```
1 | package com.miniproject.verificationApp.repository;
2 |
3 | import com.miniproject.verificationApp.model.GovernmentIdVerification;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 |
6 | import java.util.List;
7 | import java.util.Optional;
8 |
9 | public interface GovernmentIdVerificationRepository
10 |     extends JpaRepository<GovernmentIdVerification, Long> {
11 |
12 |     List<GovernmentIdVerification> findByUserId(Long userId);
13 |
14 |     Optional<GovernmentIdVerification>
15 |     findTopByUserIdAndPlaceNameAndStatusOrderByCreatedAtDesc(
16 |         Long userId,
17 |         String placeName,
18 |         GovernmentIdVerification.Status status
19 |     );
20 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/ReviewFlagRepository.

```
1 | package com.miniproject.verificationApp.repository;
2 |
3 | import com.miniproject.verificationApp.model.ReviewFlag;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 |
6 | public interface ReviewFlagRepository extends JpaRepository<ReviewFlag, Long> {
7 |
8 |     boolean existsByReviewIdAndUserId(Long reviewId, Long userId);
9 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/ReviewRepository.java

```
1 | package com.miniproject.verificationApp.repository;
2 |
3 | import com.miniproject.verificationApp.model.Review;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 |
6 | import java.util.List;
7 |
8 | public interface ReviewRepository extends JpaRepository<Review, Long> {
9 |
10 |     List<Review> findByPlaceName(String placeName);
11 |
12 |     // ■ Search by place name
13 |     List<Review> findByPlaceNameContainingIgnoreCase(String placeName);
14 |
15 |     // ■ Get all reviews by user
16 |     List<Review> findById(Long userId);
17 |
18 |     // ■ Only blockchain stored reviews for a place
19 |     List<Review> findByPlaceNameContainingIgnoreCaseAndStoredOnBlockchainTrue(
20 |         String placeName
21 |     );
22 |
23 |     // ■ My reviews only if stored on blockchain
24 |     List<Review> findByIdAndStoredOnBlockchainTrue(Long userId);
25 |
26 |     // ■ Place reviews sorted by trust points
27 |     List<Review>
28 |     findByPlaceNameContainingIgnoreCaseAndStoredOnBlockchainTrueOrderByTrustPointsDesc(
29 |         String placeName
30 |     );
31 | }
```


backend/verificationApp/src/main/java/com/miniproject/verificationApp/repository/UserRepository.java

```
1 | package com.miniproject.verificationApp.repository;
2 |
3 | import com.miniproject.verificationApp.model.User;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import java.util.Optional;
6 |
7 | public interface UserRepository extends JpaRepository<User, Long> {
8 |     Optional<User> findByEmail(String email);
9 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/AIVerificationResult.java

```
1 | package com.miniproject.verificationApp.service;
2 |
3 | public class AIVerificationResult {
4 |
5 |     private boolean verified;
6 |     private String message;
7 |     private float confidenceScore;
8 |     private String verificationType;
9 |     private String detectedLandmark;
10 |
11 |     // Private constructor
12 |     private AIVerificationResult(boolean verified, String message, float confidenceScore,
13 |                                 String verificationType, String detectedLandmark) {
14 |         this.verified = verified;
15 |         this.message = message;
16 |         this.confidenceScore = confidenceScore;
17 |         this.verificationType = verificationType;
18 |         this.detectedLandmark = detectedLandmark;
19 |     }
20 |
21 |     // Factory method for verified result
22 |     public static AIVerificationResult verified(String message, float confidence,
23 |                                                Object type, String landmark) {
24 |         return new AIVerificationResult(true, message, confidence, type.toString(), landmark);
25 |     }
26 |
27 |     // Factory method for rejected result
28 |     public static AIVerificationResult rejected(String message) {
29 |         return new AIVerificationResult(false, message, 0.0f, "REJECTED", null);
30 |     }
31 |
32 |     // Getters
33 |     public boolean isVerified() {
34 |         return verified;
35 |     }
36 |
37 |     public String getMessage() {
38 |         return message;
39 |     }
40 |
41 |     public float getConfidenceScore() {
42 |         return confidenceScore;
43 |     }
44 |
45 |     public String getVerificationType() {
46 |         return verificationType;
47 |     }
48 |
49 |     public String getDetectedLandmark() {
50 |         return detectedLandmark;
51 |     }
52 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/AIVerificationService.java

```

1 | package com.miniproject.verificationApp.service;
2 |
3 | import com.google.auth.oauth2.GoogleCredentials;
4 | import com.google.cloud.vision.v1.*;
5 | import com.google.protobuf.ByteString;
6 | import org.springframework.beans.factory.annotation.Value;
7 | import org.springframework.stereotype.Service;
8 |
9 | import java.io.File;
10 | import java.io.FileInputStream;
11 | import java.io.IOException;
12 | import java.nio.file.Files;
13 | import java.util.List;
14 |
15 | @Service
16 | public class AIVerificationService {
17 |
18 |     @Value("${google.credentials.path}")
19 |     private String credentialsPath;
20 |
21 |     public AIVerificationResult verifyProof(File imageFile, String claimedPlace) {
22 |         try {
23 |
24 |             AnnotateImageResponse response = analyzeImage(imageFile);
25 |
26 |             if (response.hasError()) {
27 |                 return AIVerificationResult.rejected(
28 |                     "Vision API Error: " + response.getError().getMessage()
29 |                 );
30 |             }
31 |
32 |             // Landmark image (photo verification)
33 |             if (!response.getLandmarkAnnotationsList().isEmpty()) {
34 |                 return verifyPhoto(response, claimedPlace);
35 |             }
36 |
37 |             // Document image (text verification)
38 |             if (!response.getTextAnnotationsList().isEmpty()) {
39 |                 return verifyDocument(response, claimedPlace);
40 |             }
41 |
42 |             return AIVerificationResult.rejected("No usable content found in image");
43 |
44 |         } catch (Exception e) {
45 |             e.printStackTrace();
46 |             return AIVerificationResult.rejected("Error: " + e.getMessage());
47 |         }
48 |     }
49 |
50 |     // ■ GOOGLE VISION AUTH FIXED
51 |     private AnnotateImageResponse analyzeImage(File imageFile) throws IOException {
52 |
53 |         GoogleCredentials credentials = GoogleCredentials
54 |             .fromStream(new FileInputStream(credentialsPath))
55 |             .createScoped(List.of("https://www.googleapis.com/auth/cloud-platform"));
56 |
57 |         ImageAnnotatorSettings settings = ImageAnnotatorSettings.newBuilder()
58 |             .setCredentialsProvider(() -> credentials)
59 |             .build();
60 |
61 |         ByteString imgBytes = ByteString.readFrom(Files.newInputStream(imageFile.toPath()));
62 |
63 |         Image img = Image.newBuilder()
64 |             .setContent(imgBytes)
65 |             .build();
66 |
67 |         Feature landmark = Feature.newBuilder()
68 |             .setType(Feature.Type.LANDMARK_DETECTION)
69 |             .build();
70 |
71 |         Feature text = Feature.newBuilder()
72 |             .setType(Feature.Type.TEXT_DETECTION)
73 |             .build();
74 |
75 |         Feature face = Feature.newBuilder()
76 |             .setType(Feature.Type.FACE_DETECTION)
77 |             .build();
78 |
79 |         AnnotateImageRequest request = AnnotateImageRequest.newBuilder()
80 |             .setImage(img)
81 |             .addFeatures(landmark)
82 |             .addFeatures(text)
83 |             .addFeatures(face)
84 |             .build();
85 |
86 |         try (ImageAnnotatorClient client = ImageAnnotatorClient.create(settings)) {
87 |
88 |             BatchAnnotateImagesResponse batch = client.batchAnnotateImages(List.of(request));
89 |

```

```

90 |         return batch.getResponses(0);
91 |     }
92 | }
93 |
94 | // =====
95 | // ■ PHOTO VERIFICATION FIXED
96 | // =====
97 | private AIVerificationResult verifyPhoto(AnnotateImageResponse response, String claimedPlace) {
98 |
99 |     if (response.getLandmarkAnnotationsList().isEmpty()) {
100 |         return AIVerificationResult.rejected("No landmark detected");
101 |     }
102 |
103 |     String detectedLandmark = response.getLandmarkAnnotationsList()
104 |         .get(0)
105 |         .getDescription();
106 |
107 |     float confidence = response.getLandmarkAnnotationsList()
108 |         .get(0)
109 |         .getScore();
110 |
111 |     if (!isMatch(detectedLandmark, claimedPlace)) {
112 |         return AIVerificationResult.rejected(
113 |             "Detected: " + detectedLandmark + " but expected: " + claimedPlace
114 |         );
115 |     }
116 |
117 |     return AIVerificationResult.verified(
118 |         "Photo verified at " + detectedLandmark,
119 |         confidence * 100,
120 |         "PHOTO",
121 |         detectedLandmark
122 |     );
123 | }
124 |
125 | // =====
126 | // ■ DOCUMENT VERIFICATION FIXED
127 | // =====
128 | private AIVerificationResult verifyDocument(AnnotateImageResponse response, String claimedPlace) {
129 |
130 |     if (response.getTextAnnotationsList().isEmpty()) {
131 |         return AIVerificationResult.rejected("No text found in document");
132 |     }
133 |
134 |     String text = response.getTextAnnotationsList()
135 |         .get(0)
136 |         .getDescription()
137 |         .toLowerCase();
138 |
139 |     if (!isMatch(text, claimedPlace)) {
140 |         return AIVerificationResult.rejected("Place not found in document");
141 |     }
142 |
143 |     return AIVerificationResult.verified(
144 |         "Document verified for " + claimedPlace,
145 |         80f,
146 |         "DOCUMENT",
147 |         claimedPlace
148 |     );
149 | }
150 |
151 | // =====
152 | // ■ SMART MATCHING LOGIC (FIX)
153 | // =====
154 | private boolean isMatch(String detected, String claimed) {
155 |
156 |     if (detected == null || claimed == null) return false;
157 |
158 |     detected = normalize(detected);
159 |     claimed = normalize(claimed);
160 |
161 |     return detected.contains(claimed) || claimed.contains(detected);
162 | }
163 |
164 | // removes brackets, symbols, extra spaces
165 | private String normalize(String text) {
166 |     return text
167 |         .toLowerCase()
168 |         .replaceAll("\\\\(.*?\\\\)", "") // remove (Agra)
169 |         .replaceAll("[^a-z0-9 ]", "") // remove symbols
170 |         .replaceAll("\\s+", " ") // multiple spaces → single
171 |         .trim();
172 | }
173 | }

```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/BlockchainService.java

```

1 | package com.miniproject.verificationApp.service;
2 |
3 | import org.springframework.stereotype.Service;
4 | import org.web3j.protocol.Web3j;
5 | import org.web3j.protocol.http.HttpService;
6 | import org.web3j.crypto.Credentials;
7 | import org.web3j.tx.RawTransactionManager;
8 | import org.web3j.tx.gas.DefaultGasProvider;
9 | import org.web3j.protocol.core.methods.response.EthSendTransaction;
10 | import org.web3j.abi.datatypes.Function;
11 | import org.web3j.abi.FunctionEncoder;
12 | import org.web3j.abi.datatypes.Utf8String;
13 | import org.web3j.abi.datatypes.Bool;
14 | import org.web3j.abi.datatypes.generated.Uint8;
15 |
16 | import java.math.BigInteger;
17 | import java.util.Arrays;
18 |
19 | @Service
20 | public class BlockchainService {
21 |
22 |     private final String INFURA_URL =
23 |         "https://sepolia.infura.io/v3/[REDACTED_INFURA_PROJECT_ID]";
24 |
25 |     // ■ YOUR METAMASK PRIVATE KEY
26 |     private final String PRIVATE_KEY =
27 |         "[REDACTED_BLOCKCHAIN_PRIVATE_KEY]";
28 |
29 |     // ■ YOUR DEPLOYED CONTRACT ADDRESS
30 |     private final String CONTRACT_ADDRESS =
31 |         "0xd9145CCE52D386f254917e481eB44e9943F39138";
32 |
33 |     public String storeHash(String hash, String place) {
34 |
35 |         try {
36 |
37 |             Web3j web3j =
38 |                 Web3j.build(new HttpService(INFURA_URL));
39 |
40 |             Credentials credentials =
41 |                 Credentials.create(PRIVATE_KEY);
42 |
43 |             RawTransactionManager txManager =
44 |                 new RawTransactionManager(web3j, credentials);
45 |
46 |             // =====
47 |             // TEMPORARY TEST VALUES
48 |             // =====
49 |
50 |             int aiScore = 94;
51 |
52 |             int nlpScore = 85;
53 |
54 |             boolean gpsVerified = true;
55 |
56 |             // =====
57 |             // SOLIDITY FUNCTION CALL
58 |             // =====
59 |
60 |             Function function = new Function(
61 |                 "storeReview",
62 |
63 |                 Arrays.asList(
64 |
65 |                     new Utf8String(hash),
66 |
67 |                     new Utf8String(place),
68 |
69 |                     new Uint8(aiScore),
70 |
71 |                     new Uint8(nlpScore),
72 |
73 |                     new Bool(gpsVerified)
74 |                 ),
75 |
76 |                 Arrays.asList()
77 |             );
78 |
79 |             String encodedFunction =
80 |                 FunctionEncoder.encode(function);
81 |
82 |             EthSendTransaction response =
83 |                 txManager.sendTransaction(
84 |
85 |                     DefaultGasProvider.GAS_PRICE,
86 |
87 |                     DefaultGasProvider.GAS_LIMIT,
88 |
89 |                     CONTRACT_ADDRESS,

```

```
90 |  
91 |         encodedFunction,  
92 |  
93 |         BigInteger.ZERO  
94 |     );  
95 |  
96 |     System.out.println("■ Transaction sent!");  
97 |  
98 |     String txHash = response.getTransactionHash();  
99 |  
100 |     System.out.println("TX HASH: " + txHash);  
101 |  
102 |     return txHash;  
103 |  
104 | } catch (Exception e) {  
105 |  
106 |     e.printStackTrace();  
107 |  
108 |     return null;  
109 | }  
110 | }  
111 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/ExifGpsService.java

```
1 | package com.miniproject.verificationApp.service;
2 |
3 | import com.drew.imaging.ImageMetadataReader;
4 | import com.drew.metadata.Metadata;
5 | import com.drew.metadata.exif.GpsDirectory;
6 | import com.drew.lang.GeoLocation;
7 |
8 | import org.springframework.stereotype.Service;
9 |
10 | import java.io.File;
11 |
12 | @Service
13 | public class ExifGpsService {
14 |
15 |     // Extract GPS coordinates from image
16 |     public double[] extractGpsCoordinates(File imageFile) {
17 |
18 |         try {
19 |
20 |             Metadata metadata =
21 |                 ImageMetadataReader.readMetadata(imageFile);
22 |
23 |             GpsDirectory gpsDirectory =
24 |                 metadata.getFirstDirectoryOfType(GpsDirectory.class);
25 |
26 |             if (gpsDirectory == null) {
27 |                 return null;
28 |             }
29 |
30 |             GeoLocation geoLocation =
31 |                 gpsDirectory.getGeoLocation();
32 |
33 |             if (geoLocation == null || geoLocation.isZero()) {
34 |                 return null;
35 |             }
36 |
37 |             return new double[]{
38 |                 geoLocation.getLatitude(),
39 |                 geoLocation.getLongitude()
40 |             };
41 |
42 |         } catch (Exception e) {
43 |             e.printStackTrace();
44 |             return null;
45 |         }
46 |     }
47 |
48 |     // Haversine Formula
49 |     public double calculateDistanceMeters(
50 |         double lat1,
51 |         double lon1,
52 |         double lat2,
53 |         double lon2
54 |     ) {
55 |
56 |         final int EARTH_RADIUS = 6371000;
57 |
58 |         double latDistance =
59 |             Math.toRadians(lat2 - lat1);
60 |
61 |         double lonDistance =
62 |             Math.toRadians(lon2 - lon1);
63 |
64 |         double a =
65 |             Math.sin(latDistance / 2)
66 |                 * Math.sin(latDistance / 2)
67 |                 + Math.cos(Math.toRadians(lat1))
68 |                 * Math.cos(Math.toRadians(lat2))
69 |                 * Math.sin(lonDistance / 2)
70 |                 * Math.sin(lonDistance / 2);
71 |
72 |         double c =
73 |             2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
74 |
75 |         return EARTH_RADIUS * c;
76 |     }
77 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/HashUtil.java

```
1 | package com.miniproject.verificationApp.service;
2 |
3 | import java.security.MessageDigest;
4 |
5 | public class HashUtil {
6 |
7 |     public static String sha256(String input) {
8 |         try {
9 |             MessageDigest md = MessageDigest.getInstance("SHA-256");
10 |             byte[] hash = md.digest(input.getBytes());
11 |
12 |             StringBuilder hex = new StringBuilder();
13 |             for (byte b : hash) {
14 |                 hex.append(String.format("%02x", b));
15 |             }
16 |             return hex.toString();
17 |
18 |         } catch (Exception e) {
19 |             throw new RuntimeException(e);
20 |         }
21 |     }
22 | }
```


backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/OCRService.java

```

1 | package com.miniproject.verificationApp.service;
2 |
3 | import org.springframework.stereotype.Service;
4 | import org.springframework.http.*;
5 | import org.springframework.util.LinkedMultiValueMap;
6 | import org.springframework.util.MultiValueMap;
7 | import org.springframework.web.client.RestTemplate;
8 | import org.json.JSONArray;
9 | import org.json.JSONObject;
10 |
11 | import java.io.File;
12 | import java.io.IOException;
13 | import java.nio.file.Files;
14 | import java.util.Base64;
15 |
16 | @Service
17 | public class OCRService {
18 |
19 |     private static final String OCR_API_URL = "https://api.ocr.space/parse/image";
20 |     private static final String OCR_API_KEY = "[REDACTED_OCR_API_KEY]"; // Replace with valid key later
21 |
22 |     public String extractText(File imageFile) {
23 |         try {
24 |             System.out.println("■ Sending file to OCR API (Base64 mode): " + imageFile.getAbsolutePath());
25 |
26 |             // Convert file to Base64
27 |             byte[] fileContent = Files.readAllBytes(imageFile.toPath());
28 |             String base64Image = Base64.getEncoder().encodeToString(fileContent);
29 |
30 |             RestTemplate restTemplate = new RestTemplate();
31 |
32 |             HttpHeaders headers = new HttpHeaders();
33 |             headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);
34 |
35 |             MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
36 |             body.add("apikey", OCR_API_KEY);
37 |             body.add("language", "eng");
38 |             body.add("isOverlayRequired", "false");
39 |             body.add("scale", "true");
40 |             body.add("isTable", "true");
41 |             body.add("base64Image", "data:image/jpeg;base64," + base64Image);
42 |
43 |             HttpEntity<MultiValueMap<String, String>> requestEntity = new HttpEntity<>(body, headers);
44 |             ResponseEntity<String> response = restTemplate.postForEntity(OCR_API_URL, requestEntity,
String.class);
45 |
46 |             String jsonResponse = response.getBody();
47 |             System.out.println("■ Raw OCR API response: " + jsonResponse);
48 |
49 |             if (jsonResponse == null || jsonResponse.isEmpty()) return "";
50 |
51 |             JSONObject json = new JSONObject(jsonResponse);
52 |             JSONArray parsedResults = json.optJSONArray("ParsedResults");
53 |             if (parsedResults != null && parsedResults.length() > 0) {
54 |                 JSONObject firstResult = parsedResults.getJSONObject(0);
55 |                 String extractedText = firstResult.optString("ParsedText", "").trim();
56 |
57 |                 System.out.println("■ Extracted text snippet: " +
58 | extractedText));
59 |
60 |                 return extractedText;
61 |             }
62 |
63 |             System.out.println("■ No ParsedResults found in OCR response.");
64 |             return "";
65 |
66 |         } catch (IOException e) {
67 |             System.err.println("■ Error reading file: " + e.getMessage());
68 |             e.printStackTrace();
69 |             return "";
70 |         } catch (Exception e) {
71 |             System.err.println("■ Error during OCR request: " + e.getMessage());
72 |             e.printStackTrace();
73 |             return "";
74 |         }
75 |     }
76 |
77 |     // ■ Validation logic for travel proof
78 |     public boolean isValidTravelProof(String text, String selectedPlace) {
79 |         if (text == null || text.isEmpty()) return false;
80 |         text = text.toLowerCase();
81 |         selectedPlace = selectedPlace.toLowerCase();
82 |
83 |         return text.contains(selectedPlace) ||
84 | text.contains("ticket") || text.contains("entry") ||
85 | text.contains("booking") || text.contains("hotel") ||
86 | text.contains("travel") || text.contains("tour") ||
87 | text.contains("guide") || text.contains("visit");

```

```
88 |     }  
89 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/service/TimeDecayService.java

```
1 | package com.miniproject.verificationApp.service;
2 |
3 | import com.miniproject.verificationApp.model.Review;
4 | import org.springframework.stereotype.Service;
5 |
6 | import java.time.Duration;
7 | import java.time.LocalDateTime;
8 | import java.util.List;
9 |
10 | @Service
11 | public class TimeDecayService {
12 |
13 |     // Controls how fast old reviews lose importance
14 |     private static final double LAMBDA = 0.05;
15 |
16 |     // -----
17 |     // RAW AVERAGE RATING
18 |     // -----
19 |     public double calculateRawRating(List<Review> reviews) {
20 |
21 |         if (reviews == null || reviews.isEmpty()) {
22 |             return 0.0;
23 |         }
24 |
25 |         double total = 0.0;
26 |
27 |         for (Review review : reviews) {
28 |             total += review.getRating();
29 |         }
30 |
31 |         return total / reviews.size();
32 |     }
33 |
34 |     // -----
35 |     // TIME-DECAY WEIGHTED RATING
36 |     // -----
37 |     public double calculateDecayedRating(List<Review> reviews) {
38 |
39 |         if (reviews == null || reviews.isEmpty()) {
40 |             return 0.0;
41 |         }
42 |
43 |         double weightedSum = 0.0;
44 |         double totalWeight = 0.0;
45 |
46 |         LocalDateTime now = LocalDateTime.now();
47 |
48 |         for (Review review : reviews) {
49 |
50 |             // ■ SAFETY CHECK (IMPORTANT FIX)
51 |             if (review.getCreatedAt() == null) {
52 |                 continue;
53 |             }
54 |
55 |             // how old the review is (in days)
56 |             long daysOld = Duration.between(
57 |                 review.getCreatedAt(),
58 |                 now
59 |             ).toDays();
60 |
61 |             // exponential decay formula
62 |             double weight = Math.exp(-LAMBDA * daysOld);
63 |
64 |             weightedSum += review.getRating() * weight;
65 |             totalWeight += weight;
66 |         }
67 |
68 |         return totalWeight == 0 ? 0.0 : weightedSum / totalWeight;
69 |     }
70 | }
```

backend/verificationApp/src/main/java/com/miniproject/verificationApp/VerificationAppApplication.java

```
1 | package com.miniproject.verificationApp;
2 |
3 | import org.springframework.boot.SpringApplication;
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
5 |
6 | @SpringBootApplication
7 | public class VerificationAppApplication {
8 |
9 |     public static void main(String[] args) {
10 |         SpringApplication.run(VerificationAppApplication.class, args);
11 |     }
12 |
13 | }
```

backend/verificationApp/src/main/resources/application.properties

```
1 | spring.application.name=verificationApp
2 |
3 | spring.datasource.url=jdbc:mysql://localhost:3306/review_system?useSSL=false&allowPublicKeyRetrieval=true&server
  | Timezone=UTC
4 | spring.datasource.username=appuser
5 | spring.datasource.password=[REDACTED_PASSWORD]
6 | spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
7 |
8 | spring.jpa.hibernate.ddl-auto=update
9 | spring.jpa.show-sql=true
10 | spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
11 |
12 | # ■ REMOVE API KEY COMPLETELY (DO NOT USE ANYMORE)
13 | # google.vision.api.key=...
14 |
15 | # File upload settings
16 | spring.servlet.multipart.max-file-size=10MB
17 | spring.servlet.multipart.max-request-size=10MB
18 | google.credentials.path=src/main/resources/google/blockchaintouristsystembASED.json
```

backend/verificationApp/src/test/java/com/miniproject/verificationApp/VerificationAppApplicationTests.java

```
1 | package com.miniproject.verificationApp;  
2 |  
3 | import org.junit.jupiter.api.Test;  
4 | import org.springframework.boot.test.context.SpringBootTest;  
5 |  
6 | @SpringBootTest  
7 | class VerificationAppApplicationTests {  
8 |  
9 |     @Test  
10 |     void contextLoads() {  
11 |     }  
12 |  
13 | }
```

frontend/package.json

```
1 | {
2 |   "name": "frontend",
3 |   "version": "0.1.0",
4 |   "private": true,
5 |   "dependencies": {
6 |     "@testing-library/dom": "^10.4.1",
7 |     "@testing-library/jest-dom": "^6.9.1",
8 |     "@testing-library/react": "^16.3.0",
9 |     "@testing-library/user-event": "^13.5.0",
10 |     "react": "^19.2.0",
11 |     "react-dom": "^19.2.0",
12 |     "react-router-dom": "^7.9.4",
13 |     "react-scripts": "5.0.1",
14 |     "react-select": "^5.10.2",
15 |     "web-vitals": "^2.1.4"
16 |   },
17 |   "scripts": {
18 |     "start": "react-scripts start",
19 |     "build": "react-scripts build",
20 |     "test": "react-scripts test",
21 |     "eject": "react-scripts eject"
22 |   },
23 |   "eslintConfig": {
24 |     "extends": [
25 |       "react-app",
26 |       "react-app/jest"
27 |     ]
28 |   },
29 |   "browserslist": {
30 |     "production": [
31 |       ">0.2%",
32 |       "not dead",
33 |       "not op_mini all"
34 |     ],
35 |     "development": [
36 |       "last 1 chrome version",
37 |       "last 1 firefox version",
38 |       "last 1 safari version"
39 |     ]
40 |   }
41 | }
```

frontend/public/index.html

```
1 | <!DOCTYPE html>
2 | <html lang="en">
3 |   <head>
4 |     <meta charset="utf-8" />
5 |     <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
6 |     <meta name="viewport" content="width=device-width, initial-scale=1" />
7 |     <meta name="theme-color" content="#000000" />
8 |     <meta
9 |       name="description"
10 |       content="Web site created using create-react-app"
11 |     />
12 |     <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
13 |     <!--
14 |       manifest.json provides metadata used when your web app is installed on a
15 |       user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
16 |     -->
17 |     <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
18 |     <!--
19 |       Notice the use of %PUBLIC_URL% in the tags above.
20 |       It will be replaced with the URL of the `public` folder during the build.
21 |       Only files inside the `public` folder can be referenced from the HTML.
22 |
23 |       Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
24 |       work correctly both with client-side routing and a non-root public URL.
25 |       Learn how to configure a non-root public URL by running `npm run build`.
26 |     -->
27 |     <title>React App</title>
28 |   </head>
29 |   <body>
30 |     <noscript>You need to enable JavaScript to run this app.</noscript>
31 |     <div id="root"></div>
32 |     <!--
33 |       This HTML file is a template.
34 |       If you open it directly in the browser, you will see an empty page.
35 |
36 |       You can add webfonts, meta tags, or analytics to this file.
37 |       The build step will place the bundled scripts into the <body> tag.
38 |
39 |       To begin the development, run `npm start` or `yarn start`.
40 |       To create a production bundle, use `npm run build` or `yarn build`.
41 |     -->
42 |   </body>
43 | </html>
```


frontend/public/manifest.json

```
1 | {
2 |   "short_name": "React App",
3 |   "name": "Create React App Sample",
4 |   "icons": [
5 |     {
6 |       "src": "favicon.ico",
7 |       "sizes": "64x64 32x32 24x24 16x16",
8 |       "type": "image/x-icon"
9 |     },
10 |    {
11 |      "src": "logo192.png",
12 |      "type": "image/png",
13 |      "sizes": "192x192"
14 |    },
15 |    {
16 |      "src": "logo512.png",
17 |      "type": "image/png",
18 |      "sizes": "512x512"
19 |    }
20 |  ],
21 |   "start_url": ".",
22 |   "display": "standalone",
23 |   "theme_color": "#000000",
24 |   "background_color": "#ffffff"
25 | }
```

frontend/public/robots.txt

```
1 | # https://www.robotstxt.org/robotstxt.html
2 | User-agent: *
3 | Disallow:
```

frontend/src/App.css

```
1 | /* Page background (FULL SCREEN MODERN LAYOUT) */
2 | body {
3 |   font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
4 |   margin: 0;
5 |   padding: 0;
6 |
7 |   background: linear-gradient(135deg, black, #f5576c);
8 |
9 |   /* BEST PRACTICE: page scroll, not card scroll */
10 |   min-height: 100vh;
11 |
12 |   display: flex;
13 |   justify-content: center;
14 |   align-items: flex-start;
15 |
16 |   padding: 60px 20px;
17 | }
18 |
19 | /* Centered modern glass card */
20 | .card {
21 |   background: rgba(255, 255, 255, 0.92);
22 |   backdrop-filter: blur(12px);
23 |
24 |   padding: 40px 30px;
25 |   border-radius: 20px;
26 |
27 |   box-shadow: 0 20px 50px rgba(0,0,0,0.25);
28 |
29 |   width: 420px;
30 |   max-width: 92%;
31 |
32 |   text-align: center;
33 |
34 |   /* IMPORTANT: NO INNER SCROLL */
35 |   overflow: visible;
36 | }
37 |
38 | /* Headings */
39 | h1, h2 {
40 |   margin-bottom: 20px;
41 |   color: #333;
42 | }
43 |
44 | /* Paragraph text */
45 | p {
46 |   margin-bottom: 20px;
47 |   color: #666;
48 | }
49 |
50 | /* Input fields */
51 | input {
52 |   width: 100%;
53 |   padding: 14px 12px;
54 |   margin: 12px 0;
55 |
56 |   border-radius: 10px;
57 |   border: 1px solid #ddd;
58 |
59 |   font-size: 15px;
60 |
61 |   transition: all 0.3s;
62 | }
63 |
64 | input:focus {
65 |   border-color: #f5576c;
66 |   box-shadow: 0 0 8px rgba(245, 87, 108, 0.4);
67 |   outline: none;
68 | }
69 |
70 | /* Buttons */
71 | button {
72 |   width: 100%;
73 |   padding: 14px;
74 |   margin: 12px 0;
75 |
76 |   border-radius: 10px;
77 |   border: none;
78 |
79 |   font-size: 16px;
80 |   cursor: pointer;
81 |
82 |   transition: all 0.3s;
83 |   color: white;
84 | }
85 |
86 | button.btn-primary {
87 |   background: linear-gradient(45deg, #f093fb, #f5576c);
88 | }
89 |
```

```
90 | button.btn-primary:hover {
91 |   opacity: 0.9;
92 |   transform: translateY(-2px);
93 | }
94 |
95 | button.btn-secondary {
96 |   background: linear-gradient(45deg, #43e97b, #38f9d7);
97 | }
98 |
99 | button.btn-secondary:hover {
100 |   opacity: 0.9;
101 |   transform: translateY(-2px);
102 | }
103 |
104 | /* Link text */
105 | .link {
106 |   color: #f5576c;
107 |   cursor: pointer;
108 |   text-decoration: underline;
109 |   font-size: 14px;
110 | }
111 |
112 | /* spacing helper */
113 | .btn-spacing {
114 |   margin-top: 20px;
115 | }
116 |
117 | /* ===== */
118 | /* DROPDOWN FIX (IMPORTANT FIX) */
119 | /* ===== */
120 |
121 | /* prevents dropdown being trapped visually */
122 | select, .dropdown, .list {
123 |   position: relative;
124 |   z-index: 10;
125 | }
126 |
127 | /* if you use custom dropdown menus */
128 | .dropdown-list {
129 |   position: absolute;
130 |   z-index: 9999;
131 |
132 |   background: white;
133 |   border-radius: 12px;
134 |
135 |   box-shadow: 0 10px 25px rgba(0,0,0,0.15);
136 | }
137 |
138 | /* ===== */
139 | /* ADDED FORM LAYOUT FIX */
140 | /* ===== */
141 |
142 | form {
143 |   display: flex;
144 |   flex-direction: column;
145 |   gap: 12px;
146 |   align-items: stretch;
147 | }
148 |
149 | textarea,
150 | select {
151 |   width: 100%;
152 |   display: block;
153 | }
```

frontend/src/App.js

```
1 | import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
2 | import Welcomepage from "./welcomepage";
3 | import Loginpage from "./loginpage";
4 | import Registrationpage from "./registrationpage";
5 | import Verificationpage from "./verificationpage";
6 | import VerificationPage2 from "./VerificationPage2";
7 | import TouristSelectPage from "./TouristSelectPage";
8 | import ReviewPage from "./Review";
9 | import Dashboard from "./Dashboard";
10 | import PlaceReviews from "./PlaceReviews";
11 | import MyReviews from "./MyReviews";
12 |
13 | function App() {
14 |   return (
15 |     <Router>
16 |       <Routes>
17 |         <Route path="/" element={<Welcomepage />} />
18 |         <Route path="/login" element={<Loginpage />} />
19 |         <Route path="/register" element={<Registrationpage />} />
20 |         <Route path="/verify" element={<Verificationpage />} />
21 |         <Route path="/verify2" element={<VerificationPage2 />} />
22 |         <Route path="/select-place" element={<TouristSelectPage />} />
23 |         <Route path="/review" element={<ReviewPage />} />
24 |         <Route path="/dashboard" element={<Dashboard />} />
25 |         <Route path="/place-reviews" element={<PlaceReviews />} />
26 |         <Route path="/my-reviews" element={<MyReviews />} />
27 |       </Routes>
28 |     </Router>
29 |   );
30 | }
31 |
32 | export default App;
```

frontend/src/App.test.js

```
1 | import { render, screen } from '@testing-library/react';
2 | import App from './App';
3 |
4 | test('renders learn react link', () => {
5 |   render(<App />);
6 |   const linkElement = screen.getByText(/learn react/i);
7 |   expect(linkElement).toBeInTheDocument();
8 | });
```

frontend/src/Dashboard.jsx

```
1 | import './App.css';
2 | import { useNavigate } from 'react-router-dom';
3 |
4 | function Dashboard() {
5 |
6 |     const navigate = useNavigate();
7 |
8 |     return (
9 |
10 |         <div className="card">
11 |
12 |             <h2>Dashboard ■</h2>
13 |
14 |             <p>Choose one of the actions below.</p>
15 |
16 |             { /* VIEW REVIEWS */ }
17 |             <button
18 |                 className="btn btn-primary"
19 |                 onClick={() => navigate("/place-reviews")}
20 |             >
21 |                 View Place Reviews
22 |             </button>
23 |             <button
24 |                 className="btn btn-secondary"
25 |                 onClick={() => navigate("/my-reviews")}
26 |             >
27 |                 View My Reviews
28 |             </button>
29 |
30 |             { /* VERIFIED REVIEW FLOW */ }
31 |             <button
32 |                 className="btn btn-secondary"
33 |                 onClick={() => navigate("/select-place")}
34 |             >
35 |                 Post a Review
36 |             </button>
37 |
38 |         </div>
39 |     );
40 | }
41 |
42 | export default Dashboard;
```

frontend/src/index.css

```
1 | body {
2 |   margin: 0;
3 |   font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
4 |     'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
5 |     sans-serif;
6 |   -webkit-font-smoothing: antialiased;
7 |   -moz-osx-font-smoothing: grayscale;
8 | }
9 |
10 | code {
11 |   font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',
12 |     monospace;
13 | }
```


frontend/src/index.js

```
1 | import React from 'react';
2 | import ReactDOM from 'react-dom/client';
3 | import './index.css';
4 | import App from './App';
5 | import reportWebVitals from './reportWebVitals';
6 |
7 | const root = ReactDOM.createRoot(document.getElementById('root'));
8 | root.render(
9 |   <React.StrictMode>
10 |     <App />
11 |   </React.StrictMode>
12 | );
13 |
14 | // If you want to start measuring performance in your app, pass a function
15 | // to log results (for example: reportWebVitals(console.log))
16 | // or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
17 | reportWebVitals();
```

frontend/src/loginpage.jsx

```

1 | import { useState } from "react";
2 | import { useNavigate } from "react-router-dom";
3 | import "../App.css";
4 |
5 | function Loginpage() {
6 |   const navigate = useNavigate();
7 |   const [email, setEmail] = useState("");
8 |   const [password, setPassword] = useState("");
9 |
10 |   const handleLogin = async (e) => {
11 |     e.preventDefault();
12 |
13 |     try {
14 |       const res = await fetch("http://localhost:8080/api/login", {
15 |         method: "POST",
16 |         headers: { "Content-Type": "application/json" },
17 |         body: JSON.stringify({
18 |           email: email,
19 |           passwordHash: password,
20 |         }),
21 |       });
22 |
23 |       const data = await res.json();
24 |
25 |       if (res.ok) {
26 |         alert(data.message);
27 |
28 |         // ■ Save email for later (for upload page or other pages)
29 |         localStorage.setItem("userEmail", email);
30 |
31 |         // ■ If email is verified → go to tourist place selection page
32 |         if (data.emailVerified) {
33 |
34 |           // save login state if needed later
35 |           localStorage.setItem("userEmail", email);
36 |
37 |           navigate("/dashboard");
38 |
39 |         } else {
40 |           // If not verified → go to email verification page
41 |           navigate("/verify");
42 |         }
43 |       } else {
44 |         alert(data.message || "Login failed");
45 |       }
46 |     } catch (err) {
47 |       console.error(err);
48 |       alert("Error connecting to backend");
49 |     }
50 |   };
51 |
52 |   return (
53 |     <div className="card">
54 |       <h2>Login ■</h2>
55 |       <form onSubmit={handleLogin}>
56 |         <input
57 |           type="email"
58 |           placeholder="Email Address"
59 |           value={email}
60 |           onChange={(e) => setEmail(e.target.value)}
61 |           required
62 |         />
63 |         <input
64 |           type="password"
65 |           placeholder="Password"
66 |           value={password}
67 |           onChange={(e) => setPassword(e.target.value)}
68 |           required
69 |         />
70 |         <button type="submit" className="btn btn-primary">
71 |           Login
72 |         </button>
73 |       </form>
74 |       <p>
75 |         Don't have an account?{" " }
76 |         <span className="link" onClick={() => navigate("/register")}>
77 |           Register
78 |         </span>
79 |       </p>
80 |     </div>
81 |   );
82 | }
83 |
84 | export default Loginpage;

```

frontend/src/logo.svg

```
1 | <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 841.9 595.3"><g fill="#61DAFB"><path d="M666.3
| 296.5c0-32.5-40.7-63.3-103.1-82.4 14.4-63.6 8-114.2-20.2-130.4-6.5-3.8-14.1-5.6-22.4-5.6v22.3c4.6 0 8.3.9 11.4
| 2.6 13.6 7.8 19.5 37.5 14.9 75.7-1.1 9.4-2.9 19.3-5.1 29.4-19.6-4.8-41-8.5-63.5-10.9-13.5-18.5-27.5-35.3-41.6-50
| 32.6-30.3 63.2-46.9 84-46.9v78c-27.5 0-63.5 19.6-99.9 53.6-36.4-33.8-72.4-53.2-99.9-53.2v22.3c20.7 0 51.4 16.5
| 84 46.6-14 14.7-28 31.4-41.3 49.9-22.6 2.4-44 6.1-63.6 11-2.3-10-4-19.7-5.2-29-4.7-38.2 1.1-67.9 14.6-75.8 3-1.8
| 6.9-2.6 11.5-2.6v78.5c-8.4 0-16 1.8-22.6 5.6-28.1 16.2-34.4 66.7-19.9 130.1-62.2 19.2-102.7 49.9-102.7 82.3 0
| 32.5 40.7 63.3 103.1 82.4-14.4 63.6-8 114.2 20.2 130.4 6.5 3.8 14.1 5.6 22.5 5.6 27.5 0 63.5-19.6 99.9-53.6 36.4
| 33.8 72.4 53.2 99.9 53.2 8.4 0 16-1.8 22.6-5.6 28.1-16.2 34.4-66.7 19.9-130.1 62-19.1 102.5-49.9
| 102.5-82.3z m-130.2-66.7c-3.7 12.9-8.3 26.2-13.5 39.5-4.1-8-8.4-16-13.1-24-4.6-8-9.5-15.8-14.4-23.4 14.2 2.1 27.9
| 4.7 41 7.9z m-45.8 106.5c-7.8 13.5-15.8 26.3-24.1 38.2-14.9 1.3-30 2-45.2 2-15.1
| 0-30.2-.7-45-1.9-8-3-11.9-16.4-24.6-24.2-38-7.6-13.1-14.5-26.4-20.8-39.8 6.2-13.4 13.2-26.8 20.7-39.9 7.8-13.5
| 15.8-26.3 24.1-38.2 14.9-1.3 30-2 45.2-2 15.1 0 30.2.7 45 1.9 8.3 11.9 16.4 24.6 24.2 38 7.6 13.1 14.5 26.4 20.8
| 39.8-6.3 13.4-13.2 26.8-20.7 39.9z m32.3-13c5.4 13.4 10 26.8 13.8 39.8-13.1 3.2-26.9 5.9-41.2 8 4.9-7.7 9.8-15.6
| 14.4-23.7 4.6-8 8.9-16.1 13-24.1z m421.2 430c-9.3-9.6-18.6-20.3-27.8-32 9.4 18.2.7 27.5.7 9.4 0 18.7-.2
| 27.8-.7-9 11.7-18.3 22.4-27.5 32z m-74.4-58.9c-14.2-2.1-27.9-4.7-41-7.9 3.7-12.9 8.3-26.2 13.5-39.5 4.1 8 8.4 16
| 13.1 24 4.7 8 9.5 15.8 14.4 23.4z m420.7 163c9.3 9.6 18.6 20.3 27.8 32-9.4-18.2-.7-27.5-.7-9.4 0 18.7.2-27.8.7
| 9-11.7 18.3-22.4 27.5-32z m-74 58.9c-4.9 7.7-9.8 15.6-14.4 23.7-4.6 8-8.9 16-13 24-5.4-13.4-10-26.8-13.8-39.8
| 13.1-3.1 26.9-5.8 41.2-7.9z m-90.5 125.2c-35.4-15.1-58.3-34.9-58.3-50.6 0-15.7 22.9-35.6 58.3-50.6 8.6-3.7 18-7
| 27.7-10.1 5.7 19.6 13.2 40 22.5 60.9-9.2 20.8-16.6 41.1-22.2 60.6-9.9-3.1-19.3-6.5-28-10.2z m310
| 490c-13.6-7.8-19.5-37.5-14.9-75.7 1.1-9.4 2.9-19.3 5.1-29.4 19.6 4.8 41 8.5 63.5 10.9 13.5 18.5 27.5 35.3 41.6
| 50-32.6 30.3-63.2 46.9-84 46.9-4.5-.1-8.3-1-11.3-2.7z m237.2-76.2c4.7 38.2-1.1 67.9-14.6 75.8-3 1.8-6.9 2.6-11.5
| 2.6-20.7 0-51.4-16.5-84-46.6 14-14.7 28-31.4 41.3-49.9 22.6-2.4 44-6.1 63.6-11 2.3 10.1 4.1 19.8 5.2
| 29.1z m38.5-66.7c-8.6 3.7-18 7-27.7 10.1-5.7-19.6-13.2-40-22.5-60.9 9.2-20.8 16.6-41.1 22.2-60.6 9.9 3.1 19.3 6.5
| 28.1 10.2 35.4 15.1 58.3 34.9 58.3 50.6-.1 15.7-23 35.6-58.4 50.6z m320.8 78.4z"/><circle cx="420.9" cy="296.5"
| r="45.7"/><path d="M520.5 78.1z"/></g></svg>
```

frontend/src/MyReviews.jsx

```

1 | import React, { useEffect, useState } from "react";
2 | import { useNavigate } from "react-router-dom";
3 | import "./App.css";
4 |
5 | export default function MyReviews() {
6 |
7 |     const [reviews, setReviews] = useState([]);
8 |     const navigate = useNavigate();
9 |
10 |    const userEmail = localStorage.getItem("userEmail");
11 |
12 |    const fetchMyReviews = async () => {
13 |        try {
14 |            const res = await fetch(
15 |                `http://localhost:8080/api/reviews/my-reviews?email=${userEmail}`
16 |            );
17 |
18 |            const data = await res.json();
19 |            setReviews(data);
20 |
21 |        } catch (error) {
22 |            console.error("Error fetching my reviews:", error);
23 |        }
24 |    };
25 |
26 |    useEffect(() => {
27 |        fetchMyReviews();
28 |    }, []);
29 |
30 |    return (
31 |        <div className="card">
32 |            <h2>My Posted Reviews <span>▲</span></h2>
33 |
34 |            {reviews.length === 0 ? (
35 |                <p>You have not posted any reviews yet.</p>
36 |            ) : (
37 |                <ul style={{ listStyle: "none", padding: 0 }}>
38 |                    {reviews.map((rev) => (
39 |                        <li
40 |                            key={rev.id}
41 |                            style={{
42 |                                background: "#f5f5f5",
43 |                                padding: "15px",
44 |                                marginBottom: "12px",
45 |                                borderRadius: "10px",
46 |                            }}
47 |                        >
48 |                            <h4>■ {rev.placeName}</h4>
49 |                            <p>■ Rating: {rev.rating}</p>
50 |                            <p>{rev.reviewText}</p>
51 |
52 |                            {rev.isDisputed && (
53 |                                <p style={{ color: "red", fontWeight: "bold" }}>
54 |                                    ■ DISPUTED REVIEW
55 |                                </p>
56 |                            )}
57 |                        </li>
58 |                    )}
59 |                </ul>
60 |            )}
61 |
62 |            <button
63 |                onClick={() => navigate("/dashboard")}
64 |                className="btn btn-secondary"
65 |            >
66 |                ■ Back to Dashboard
67 |            </button>
68 |        </div>
69 |    );
70 | }

```

frontend/src/PlaceReviews.jsx

```
1 | import React, { useEffect, useState } from "react";
2 | import { useNavigate } from "react-router-dom";
3 | import "../App.css";
4 |
5 | export default function PlaceReviews() {
6 |
7 |     const navigate = useNavigate();
8 |
9 |     const [reviews, setReviews] = useState([]);
10 |    const [filteredReviews, setFilteredReviews] = useState([]);
11 |    const [searchPlace, setSearchPlace] = useState("");
12 |
13 |    const userEmail = localStorage.getItem("userEmail");
14 |
15 |    // =====
16 |    // FETCH ALL REVIEWS
17 |    // =====
18 |
19 |    const fetchReviews = async () => {
20 |
21 |        try {
22 |
23 |            const res = await fetch(
24 |                "http://localhost:8080/api/reviews/all"
25 |            );
26 |
27 |            const data = await res.json();
28 |
29 |            setReviews(data);
30 |            setFilteredReviews([]);
31 |
32 |        } catch (error) {
33 |
34 |            console.error("Error fetching reviews:", error);
35 |        }
36 |    };
37 |
38 |    useEffect(() => {
39 |
40 |        fetchReviews();
41 |
42 |    }, []);
43 |
44 |    // =====
45 |    // SEARCH BY PLACE
46 |    // =====
47 |
48 |    const handleSearch = async () => {
49 |
50 |        try {
51 |
52 |            const res = await fetch(
53 |                `http://localhost:8080/api/reviews/search?place=${searchPlace}`
54 |            );
55 |
56 |            const data = await res.json();
57 |
58 |            setFilteredReviews(data);
59 |
60 |        } catch (error) {
61 |
62 |            console.error("Search error:", error);
63 |        }
64 |    };
65 |
66 |    // =====
67 |    // FLAG REVIEW
68 |    // =====
69 |
70 |    const handleFlag = async (reviewId) => {
71 |
72 |        try {
73 |
74 |            const res = await fetch(
75 |                "http://localhost:8080/api/reviews/flag",
76 |                {
77 |                    method: "POST",
78 |                    headers: {
79 |                        "Content-Type": "application/json",
80 |                    },
81 |                    body: JSON.stringify({
82 |                        reviewId: reviewId,
83 |                        email: userEmail,
84 |                    }),
85 |                }
86 |            );
87 |
88 |            const text = await res.text();
89 |
```

```
90 |         alert(text);
91 |
92 |         handleSearch();
93 |
94 |     } catch (error) {
95 |
96 |         console.error(error);
97 |
98 |         alert("Error flagging review");
99 |     }
100 | };
101 |
102 | return (
103 |
104 |     <div className="card">
105 |
106 |         <h2>Place Reviews ■■</h2>
107 |
108 |         { /* SEARCH */ }
109 |         <div style={{ marginBottom: "20px" }}>
110 |
111 |             <input
112 |                 type="text"
113 |                 placeholder="Enter tourist place..."
114 |                 value={searchPlace}
115 |                 onChange={(e) =>
116 |                     setSearchPlace(e.target.value)
117 |                 }
118 |             />
119 |
120 |             <button
121 |                 className="btn btn-primary"
122 |                 onClick={handleSearch}
123 |             >
124 |                 Search
125 |             </button>
126 |
127 |         </div>
128 |
129 |         { /* REVIEWS */ }
130 |
131 |         {filteredReviews.length === 0 ? (
132 |
133 |             <p>No reviews found.</p>
134 |
135 |         ) : (
136 |
137 |             <ul style={{ listStyle: "none", padding: 0 }}>
138 |
139 |                 {filteredReviews.map((rev) => (
140 |
141 |                     <li
142 |                         key={rev.id}
143 |                         style={{
144 |                             background: "#f5f5f5",
145 |                             padding: "15px",
146 |                             marginBottom: "12px",
147 |                             borderRadius: "10px",
148 |                         }}
149 |                     >
150 |
151 |                         <h4>
152 |                             ■ {rev.placeName}
153 |                         </h4>
154 |
155 |                         <p>
156 |                             ■ Rating: {rev.rating}
157 |                         </p>
158 |
159 |                         <p>
160 |                             {rev.reviewText}
161 |                         </p>
162 |
163 |                         { /* DISPUTED */ }
164 |                         {rev.isDisputed && (
165 |
166 |                             <p
167 |                                 style={{
168 |                                     color: "red",
169 |                                     fontWeight: "bold",
170 |                                 }}
171 |                             >
172 |                                 ■■ DISPUTED REVIEW
173 |                             </p>
174 |                         )}
175 |
176 |                         { /* FLAG BUTTON */ }
177 |
178 |                         <button
179 |                             onClick={() =>
180 |                                 handleFlag(rev.id)
```

```
181 |         }
182 |         style={{
183 |           backgroundColor: "#ff4d4d",
184 |           color: "white",
185 |           border: "none",
186 |           padding: "8px 12px",
187 |           borderRadius: "8px",
188 |           cursor: "pointer",
189 |         }}
190 |       >
191 |         ■ Flag Review
192 |       </button>
193 |
194 |     </li>
195 |   )}}
196 |
197 | </ul>
198 | })
199 |
200 | { /* BACK BUTTON */}
201 |
202 | <button
203 |   onClick={() => navigate("/dashboard")}
204 |   className="btn btn-secondary"
205 | >
206 |   ■ Back to Dashboard
207 | </button>
208 |
209 | </div>
210 | );
211 | }
```

frontend/src/registrationpage.jsx

```
1 | import { useState } from "react";
2 | import { useNavigate } from "react-router-dom";
3 | import "./App.css";
4 |
5 | function Registrationpage() {
6 |   const navigate = useNavigate();
7 |   const [name, setName] = useState("");
8 |   const [email, setEmail] = useState("");
9 |   const [password, setPassword] = useState("");
10 |
11 |   const handleRegister = async (e) => {
12 |     e.preventDefault();
13 |     try {
14 |       const res = await fetch("http://localhost:8080/api/register", {
15 |         method: "POST",
16 |         headers: { "Content-Type": "application/json" },
17 |         body: JSON.stringify({
18 |           name: name,
19 |           email: email,
20 |           passwordHash: password // match backend field
21 |         }),
22 |       });
23 |
24 |       const data = await res.text(); // backend returns a plain string
25 |
26 |       if (res.ok) {
27 |         alert(data); // show backend message
28 |         navigate("/verify", { state: { email } });
29 |       } else {
30 |         alert(data);
31 |       }
32 |     } catch (err) {
33 |       console.error(err);
34 |       alert("Error connecting to backend");
35 |     }
36 |   };
37 |
38 |   return (
39 |     <div className="card">
40 |       <h2>Register ■</h2>
41 |       <form onSubmit={handleRegister}>
42 |         <input
43 |           type="text"
44 |           placeholder="Full Name"
45 |           value={name}
46 |           onChange={(e) => setName(e.target.value)}
47 |           required
48 |         />
49 |         <input
50 |           type="email"
51 |           placeholder="Email Address"
52 |           value={email}
53 |           onChange={(e) => setEmail(e.target.value)}
54 |           required
55 |         />
56 |         <input
57 |           type="password"
58 |           placeholder="Password"
59 |           value={password}
60 |           onChange={(e) => setPassword(e.target.value)}
61 |           required
62 |         />
63 |         <button type="submit" className="btn btn-primary">Register</button>
64 |       </form>
65 |       <p>
66 |         Already have an account? <span className="link" onClick={() => navigate("/login")}>Login</span>
67 |       </p>
68 |     </div>
69 |   );
70 | }
71 |
72 | export default Registrationpage;
```


frontend/src/reportWebVitals.js

```
1 | const reportWebVitals = onPerfEntry => {  
2 |   if (onPerfEntry && onPerfEntry instanceof Function) {  
3 |     import('web-vitals').then(({ getCLS, getFID, getFCP, getLCP, getTTFB }) => {  
4 |       getCLS(onPerfEntry);  
5 |       getFID(onPerfEntry);  
6 |       getFCP(onPerfEntry);  
7 |       getLCP(onPerfEntry);  
8 |       getTTFB(onPerfEntry);  
9 |     });  
10 |   }  
11 | };  
12 |  
13 | export default reportWebVitals;
```

frontend/src/Review.jsx

```

1 | import React, { useState, useEffect } from "react";
2 | import { useNavigate } from "react-router-dom";
3 | import "./App.css";
4 |
5 | export default function ReviewPage() {
6 |   const navigate = useNavigate();
7 |   const [review, setReview] = useState("");
8 |   const [reviews, setReviews] = useState([]);
9 |   const [rating, setRating] = useState(5);
10 |   const [status, setStatus] = useState("");
11 |   const userEmail = localStorage.getItem("userEmail");
12 |
13 |   const fetchStatus = async () => {
14 |     try {
15 |       const res = await fetch(
16 |         `http://localhost:8080/api/verification/status?email=${userEmail}`
17 |       );
18 |       const data = await res.json();
19 |       if (data.length > 0) {
20 |         setStatus(data[data.length - 1].status);
21 |       }
22 |     } catch (error) {
23 |       console.error("Error fetching status:", error);
24 |     }
25 |   };
26 |
27 |   const fetchReviews = async () => {
28 |     const res = await fetch("http://localhost:8080/api/reviews/all");
29 |     const data = await res.json();
30 |     setReviews(data);
31 |   };
32 |
33 |   useEffect(() => {
34 |     fetchReviews();
35 |   }, []);
36 |
37 |   useEffect(() => {
38 |     if (userEmail) fetchStatus();
39 |   }, []);
40 |
41 |   const handleAddReview = async (e) => {
42 |     e.preventDefault();
43 |
44 |     if ((status || "").toLowerCase() !== "verified") {
45 |       alert("You can post a review only after verification!");
46 |       return;
47 |     }
48 |
49 |     try {
50 |       const res = await fetch("http://localhost:8080/api/reviews/add", {
51 |         method: "POST",
52 |         headers: { "Content-Type": "application/json" },
53 |         body: JSON.stringify({
54 |           email: userEmail,
55 |           review: review,
56 |           place: localStorage.getItem("selectedPlace"),
57 |           rating: rating
58 |         }),
59 |       });
60 |
61 |       const text = await res.text();
62 |       alert(text);
63 |       setReview("");
64 |       fetchReviews();
65 |
66 |     } catch (err) {
67 |       console.error(err);
68 |       alert("Error saving review");
69 |     }
70 |   };
71 |
72 |   return (
73 |     <div className="card" style={{ padding: "25px" }}>
74 |       <h2 style={{ marginBottom: "10px" }}>Share Your Experience ■</h2>
75 |
76 |       <p>
77 |         Status:{ " " }
78 |         <strong style={{
79 |           color:
80 |             (status || "").toLowerCase() === "verified"
81 |               ? "green"
82 |               : "red"
83 |         }}>
84 |           {status || "Not Verified"}
85 |         </strong>
86 |       </p>
87 |
88 |       { /* ===== FORM ===== */ }
89 |       <form onSubmit={handleAddReview}>

```

```

90 |         <select
91 |             value={rating}
92 |             onChange={(e) => setRating(Number(e.target.value))}
93 |             style={{
94 |                 width: "100%",
95 |                 padding: "10px",
96 |                 marginBottom: "10px",
97 |                 borderRadius: "8px"
98 |             }}
99 |         >
100 |             <option value="1">1 Star</option>
101 |             <option value="2">2 Stars</option>
102 |             <option value="3">3 Stars</option>
103 |             <option value="4">4 Stars</option>
104 |             <option value="5">5 Stars</option>
105 |         </select>
106 |
107 |         <textarea
108 |             placeholder="Write your review..."
109 |             value={review}
110 |             onChange={(e) => setReview(e.target.value)}
111 |             rows="4"
112 |             style={{
113 |                 width: "100%",
114 |                 padding: "10px",
115 |                 borderRadius: "8px",
116 |                 marginBottom: "10px"
117 |             }}
118 |         />
119 |
120 |         {/ * ■ FIXED ADD REVIEW BUTTON */}
121 |         <button
122 |             type="submit"
123 |             style={{
124 |                 width: "100%",
125 |                 padding: "12px",
126 |                 backgroundColor: "#4CAF50",
127 |                 color: "white",
128 |                 border: "none",
129 |                 borderRadius: "10px",
130 |                 fontSize: "16px",
131 |                 cursor: "pointer"
132 |             }}
133 |         >
134 |             ■ Add Review
135 |         </button>
136 |     </form>
137 |
138 |     {/ * ===== REVIEWS ===== */}
139 |     <h3 style={{ marginTop: "25px" }}>All Reviews</h3>
140 |
141 |     <ul style={{ listStyle: "none", padding: 0 }}>
142 |         {reviews.map((rev) => (
143 |             <li
144 |                 key={rev.id}
145 |                 style={{
146 |                     background: "#f7f7f7",
147 |                     padding: "12px",
148 |                     marginBottom: "10px",
149 |                     borderRadius: "10px"
150 |                 }}
151 |             >
152 |                 ■ {rev.rating} - {rev.reviewText} ({rev.placeName})
153 |
154 |                 {rev.isDisputed && (
155 |                     <span style={{
156 |                         color: "red",
157 |                         marginLeft: "10px",
158 |                         fontWeight: "bold"
159 |                     }}>
160 |                         ■■ DISPUTED
161 |                     </span>
162 |                 )}
163 |
164 |                 {/ * ■ FIXED FLAG BUTTON */}
165 |                 <button
166 |                     onClick={async () => {
167 |                         const res = await fetch(
168 |                             "http://localhost:8080/api/reviews/flag",
169 |                             {
170 |                                 method: "POST",
171 |                                 headers: {
172 |                                     "Content-Type": "application/json"
173 |                                 },
174 |                                 body: JSON.stringify({
175 |                                     reviewId: rev.id,
176 |                                     email: userEmail
177 |                                 })
178 |                             }
179 |                         );
180 |

```

```
181 |         const text = await res.text();
182 |         alert(text);
183 |         fetchReviews();
184 |     }}
185 |     style={{
186 |         marginTop: "8px",
187 |         marginLeft: "10px",
188 |         padding: "6px 10px",
189 |         backgroundColor: "#ff4d4d",
190 |         color: "white",
191 |         border: "none",
192 |         borderRadius: "6px",
193 |         cursor: "pointer"
194 |     }}
195 |     >
196 |         ■ Flag
197 |     </button>
198 | </li>
199 |   )}}
200 | </ul>
201 |
202 |   <button
203 |     onClick={() => navigate("/verify2")}
204 |     style={{
205 |       marginTop: "20px",
206 |       width: "100%",
207 |       padding: "10px",
208 |       backgroundColor: "#333",
209 |       color: "white",
210 |       borderRadius: "10px",
211 |       border: "none"
212 |     }}
213 |   >
214 |     ■ Back
215 |   </button>
216 | </div>
217 | );
218 | }
```

frontend/src/setupTests.js

```
1 | // jest-dom adds custom jest matchers for asserting on DOM nodes.  
2 | // allows you to do things like:  
3 | // expect(element).toHaveTextContent(/react/i)  
4 | // learn more: https://github.com/testing-library/jest-dom  
5 | import '@testing-library/jest-dom';
```

frontend/src/TouristSelectPage.jsx

```

1 | import React, { useState, useEffect } from "react";
2 | import { useNavigate } from "react-router-dom";
3 | import Select from "react-select";
4 | import "../App.css";
5 |
6 | export default function TouristSelectPage() {
7 |   const navigate = useNavigate();
8 |
9 |   const [inputValue, setInputValue] = useState("");
10 |   const [options, setOptions] = useState([]);
11 |   const [selectedPlace, setSelectedPlace] = useState(null);
12 |
13 |   // ■ Fetch from Nominatim
14 |   const fetchPlaces = async (query) => {
15 |     if (query.length < 3) return;
16 |
17 |     try {
18 |       const response = await fetch(
19 |         `https://nominatim.openstreetmap.org/search?q=${query}&format=json&addressdetails=1&limit=5`,
20 |         {
21 |           headers: {
22 |             "User-Agent": "tourist-app (subhashini10.3.2005@gmail.com)",
23 |           },
24 |         }
25 |       );
26 |
27 |       const data = await response.json();
28 |
29 |       const formatted = data.map((place) => ({
30 |         label: place.display_name,
31 |         value: place.display_name,
32 |         lat: place.lat,
33 |         lon: place.lon,
34 |       }));
35 |
36 |       setOptions(formatted);
37 |     } catch (error) {
38 |       console.error("Error fetching places:", error);
39 |     }
40 |   };
41 |
42 |   // ■ Debounce (VERY IMPORTANT)
43 |   useEffect(() => {
44 |     const delay = setTimeout(() => {
45 |       fetchPlaces(inputValue);
46 |     }, 500); // 500ms delay
47 |
48 |     return () => clearTimeout(delay);
49 |   }, [inputValue]);
50 |
51 |   const handleSubmit = (e) => {
52 |     e.preventDefault();
53 |
54 |     if (!selectedPlace) {
55 |       alert("Please select a tourist place!");
56 |       return;
57 |     }
58 |
59 |     localStorage.setItem("selectedPlace", selectedPlace.value);
60 |     localStorage.setItem("selectedLat", selectedPlace.lat);
61 |     localStorage.setItem("selectedLon", selectedPlace.lon);
62 |     navigate("/verify2");
63 |   };
64 |
65 |   return (
66 |     <div className="card">
67 |       <h2>Select Tourist Place ■■</h2>
68 |
69 |       <form onSubmit={handleSubmit}>
70 |         <Select
71 |           options={options}
72 |           value={selectedPlace}
73 |           onChange={setSelectedPlace}
74 |           onChange={value => setInputValue(value)}
75 |           placeholder="Start typing a place..."
76 |           isSearchable
77 |           noOptionsMessage={() => "Type at least 3 letters"}
78 |         />
79 |
80 |         <button type="submit" className="btn btn-primary">
81 |           Continue
82 |         </button>
83 |       </form>
84 |     </div>
85 |   );
86 | }

```

frontend/src/verificationpage.jsx

```
1 | import { useState } from "react";
2 | import { useLocation, useNavigate } from "react-router-dom";
3 | import "../App.css";
4 |
5 | function Verificationpage() {
6 |   const navigate = useNavigate();
7 |   const location = useLocation();
8 |   const email = location.state?.email || "";
9 |
10 |   const [token, setToken] = useState("");
11 |
12 |   const handleSendToken = async () => {
13 |     try {
14 |       const res = await fetch("http://localhost:8080/api/send-token", {
15 |         method: "POST",
16 |         headers: { "Content-Type": "application/json" },
17 |         body: JSON.stringify({ email }),
18 |       });
19 |       const data = await res.text();
20 |       alert(data);
21 |     } catch (err) {
22 |       console.error(err);
23 |       alert("Error sending token");
24 |     }
25 |   };
26 |
27 |   const handleVerify = async (e) => {
28 |     e.preventDefault();
29 |     try {
30 |       const res = await fetch("http://localhost:8080/api/verify-token", {
31 |         method: "POST",
32 |         headers: { "Content-Type": "application/json" },
33 |         body: JSON.stringify({ email, token }),
34 |       });
35 |       const data = await res.text();
36 |       alert(data);
37 |       if (res.ok) navigate("/login");
38 |     } catch (err) {
39 |       console.error(err);
40 |       alert("Error verifying token");
41 |     }
42 |   };
43 |
44 |   return (
45 |     <div className="card">
46 |       <h2>Email Verification ■</h2>
47 |       <p>Email: {email}</p>
48 |       <button onClick={handleSendToken} className="btn btn-secondary">
49 |         Send Verification Token
50 |       </button>
51 |       <form onSubmit={handleVerify}>
52 |         <input
53 |           type="text"
54 |           placeholder="Enter Token"
55 |           value={token}
56 |           onChange={(e) => setToken(e.target.value)}
57 |           required
58 |         />
59 |         <button type="submit" className="btn btn-primary">Verify Email</button>
60 |       </form>
61 |     </div>
62 |   );
63 | }
64 |
65 | export default Verificationpage;
```

frontend/src/VerificationPage2.jsx

```
1 | import React, { useState, useEffect } from "react";
2 | import { useNavigate } from "react-router-dom";
3 | import "../App.css";
4 |
5 | export default function VerificationPage2() {
6 |
7 |     const [file, setFile] = useState(null);
8 |     const [verifications, setVerifications] = useState([]);
9 |
10 |    const userEmail = localStorage.getItem("userEmail");
11 |
12 |    const navigate = useNavigate();
13 |
14 |    // =====
15 |    // FETCH VERIFICATION STATUS
16 |    // =====
17 |
18 |    const fetchStatus = async () => {
19 |
20 |        try {
21 |
22 |            const res = await fetch(
23 |                `http://localhost:8080/api/verification/status?email=${userEmail}`
24 |            );
25 |
26 |            const data = await res.json();
27 |
28 |            setVerifications(data);
29 |
30 |        } catch (error) {
31 |
32 |            console.error("Error fetching status:", error);
33 |        }
34 |    };
35 |
36 |    // =====
37 |    // HANDLE FILE UPLOAD
38 |    // =====
39 |
40 |    const handleUpload = async (e) => {
41 |
42 |        e.preventDefault();
43 |
44 |        if (!file) {
45 |            return alert("Please choose a file!");
46 |        }
47 |
48 |        const formData = new FormData();
49 |
50 |        formData.append("email", userEmail);
51 |        formData.append("file", file);
52 |
53 |        formData.append(
54 |            "selectedPlace",
55 |            localStorage.getItem("selectedPlace")
56 |        );
57 |
58 |        formData.append(
59 |            "selectedLat",
60 |            localStorage.getItem("selectedLat")
61 |        );
62 |
63 |        formData.append(
64 |            "selectedLon",
65 |            localStorage.getItem("selectedLon")
66 |        );
67 |
68 |        try {
69 |
70 |            // Loading message
71 |            alert("■ AI is analyzing your image... Please wait.");
72 |
73 |            const res = await fetch(
74 |                "http://localhost:8080/api/verification/upload",
75 |                {
76 |                    method: "POST",
77 |                    body: formData,
78 |                }
79 |            );
80 |
81 |            const text = await res.text();
82 |
83 |            // =====
84 |            // VERIFIED RESPONSE
85 |            // =====
86 |
87 |            if (text.startsWith("VERIFIED:")) {
88 |
89 |                const parts = text.split("|");
```



```

90 |
91 |         const place =
92 |             parts[0].split(":")[1];
93 |
94 |         const confidence =
95 |             parts[1]?.split(":")[1] || "N/A";
96 |
97 |         const type =
98 |             parts[2]?.split(":")[1] || "N/A";
99 |
100 |         const gps =
101 |             parts[3]?.split(":")[1] || "false";
102 |
103 |         const distance =
104 |             parts[4]?.split(":")[1] || "N/A";
105 |
106 |         alert(
107 |             `■ Proof verified for ${place}!
108 |
109 |             ■ AI Confidence: ${confidence}%
110 |             ■ Verification Type: ${type}
111 |             ■ GPS Verified: ${gps}
112 |             ■ Distance: ${distance} meters`
113 |         );
114 |
115 |         localStorage.setItem(
116 |             "verifiedPlace",
117 |             place
118 |         );
119 |
120 |         navigate("/review");
121 |     }
122 |
123 |     // =====
124 |     // REJECTED RESPONSE
125 |     // =====
126 |
127 |     else if (text.startsWith("REJECTED:")) {
128 |
129 |         const parts = text.split("|");
130 |
131 |         const reason =
132 |             parts[0]?.replace(
133 |                 "REJECTED:",
134 |                 ""
135 |             ) || "Unknown";
136 |
137 |         const confidence =
138 |             parts[1]?.split(":")[1] || "N/A";
139 |
140 |         const gpsReason =
141 |             parts[2]?.split(":")[1] || "No GPS info";
142 |
143 |         alert(
144 |             `■ Verification Failed
145 |
146 |             ■ Reason:
147 |             ${reason}
148 |
149 |             ■ AI Confidence:
150 |             ${confidence}%
151 |
152 |             ■ GPS Status:
153 |             ${gpsReason}
154 |
155 |             Please upload a clearer and valid travel proof image.`
156 |         );
157 |     }
158 |
159 |     // =====
160 |     // OTHER / UNKNOWN RESPONSE
161 |     // =====
162 |
163 |     else {
164 |
165 |         alert(text);
166 |     }
167 |
168 |     // Refresh verification history
169 |     fetchStatus();
170 |
171 | } catch (error) {
172 |
173 |     alert("Upload failed. Please try again.");
174 |
175 |     console.error(error);
176 | }
177 | };
178 |
179 | // =====
180 | // LOAD STATUS ON PAGE LOAD

```

```

181 | // =====
182 |
183 | useEffect(() => {
184 |
185 |     if (userEmail) {
186 |
187 |         fetchStatus();
188 |     }
189 |
190 | }, []);
191 |
192 | // =====
193 | // UI
194 | // =====
195 |
196 | return (
197 |
198 |     <div className="card">
199 |
200 |         <h2>Account Verification ■</h2>
201 |
202 |         <p>
203 |             Selected Place:{ " "}
204 |             <strong>
205 |                 {localStorage.getItem("selectedPlace") || "None"}
206 |             </strong>
207 |         </p>
208 |
209 |         <form onSubmit={handleUpload}>
210 |
211 |             <input
212 |                 type="file"
213 |                 accept="image/*,application/pdf"
214 |                 onChange={(e) =>
215 |                     setFile(e.target.files[0])
216 |                 }
217 |                 required
218 |             />
219 |
220 |             <p
221 |                 style={{
222 |                     fontSize: "0.9em",
223 |                     color: "#666",
224 |                     marginTop: "8px",
225 |                 }}
226 |             >
227 |                 Upload travel proof for the selected tourist place.
228 |             </p>
229 |
230 |             <button
231 |                 type="submit"
232 |                 className="btn btn-primary"
233 |             >
234 |                 Upload Proof
235 |             </button>
236 |
237 |         </form>
238 |
239 |         <h3 style={{ marginTop: "25px" }}>
240 |             Your Verifications
241 |         </h3>
242 |
243 |         {verifications.length === 0 ? (
244 |
245 |             <p>No verifications yet.</p>
246 |
247 |         ) : (
248 |
249 |             <ul>
250 |
251 |                 {verifications.map((v, index) => (
252 |
253 |                     <li key={index}>
254 |
255 |                         <strong>
256 |                             {v.placeName || "Unknown Place"}:
257 |                         </strong>{ " "}
258 |
259 |                         <span
260 |                             style={{
261 |                                 color:
262 |                                     v.status === "VERIFIED"
263 |                                     ? "green"
264 |                                     : v.status === "REJECTED"
265 |                                     ? "red"
266 |                                     : "gray",
267 |                             }}
268 |                         >
269 |                             {v.status}
270 |                         </span>
271 |

```

```
272 |         </li>
273 |
274 |         )}}
275 |
276 |     </ul>
277 |   )}
278 |
279 |   <button
280 |     onClick={() => navigate("/select-place")}
281 |     className="btn btn-secondary"
282 |     style={{ marginTop: "15px" }}
283 |   >
284 |     Select Tourist Place
285 |   </button>
286 |
287 | </div>
288 | );
289 | }
```

frontend/src/welcomepage.jsx

```
1 | import { useNavigate } from "react-router-dom";
2 | import "./App.css";
3 | function Welcomepage() {
4 |   const navigate = useNavigate();
5 |
6 |   return (
7 |     <div className="card">
8 |       <h1>Welcome to Review System ■</h1>
9 |       <p>Please choose an option below:</p>
10 |       <button className="btn btn-primary" onClick={() => navigate("/register")}>
11 |         Register
12 |       </button>
13 |       <button className="btn btn-secondary" onClick={() => navigate("/login")}>
14 |         Login
15 |       </button>
16 |     </div>
17 |   );
18 | }
19 |
20 | export default Welcomepage;
```